

Looped_Transformer

2026 年 5 月 17 日

1 Looped Transformer

1.1 理论

Looped Transformer 所有层共享相同的权重 θ ，每层的输入是前一层输出与初始输入的和：

$$\begin{aligned} h_0 &= \text{input} \\ h_l &= \text{TransformerBlock}(h_{l-1} + h_0 | \theta) \quad \text{for } l = 1, 2, \dots, L \end{aligned}$$

1.1.1 截断损失函数

$$\text{Loss}(\theta) = \mathbb{E}_P \left[\frac{1}{b - b_0} \sum_{t=b_0}^b \frac{1}{k+1} \sum_{i=0}^k (Y_t(P^i | \theta) - f(x_{i+1}))^2 \right]$$

其中 - $\min_{\theta}$ ：这是我们在寻找一组最优的物理定律（模型参数 θ ）。 - $\mathbb{E}_P$ ：对所有生成的 Prompt 分布求期望。在统计物理中，这相当于求“系综平均”，即我们在无数种不同的初始条件（数据集）下测试这个定律是否普适。 - $\frac{1}{b-b_0} \sum_{t=b_0}^b$ ：这是时间平均。 b 是总迭代次数， T 是窗口大小， $b_0 = \max(b - T, 0)$ 。我们只关心系统演化到“稳态”附近的这段时间内（也就是 b_0 到 b 的表现），前面瞬态的误差我们不计较。 - $\frac{1}{k+1} \sum_{i=0}^k$ ：这是空间/序列平均。也就是在给定不同长度的上下文 P^i 时，模型预测下一个点的值与真实函数 $f(x_{i+1})$ 之间的均方误差。 - $Y_t(P^i | \theta)$ ：在时间 t 、给定历史信息 P^i 和系统参数 θ 时，系统当前的状态输出。

1.1.2 下游任务

- 数据生成方式：这篇论文解决的是纯粹的数学数据拟合问题。数据集是在训练循环中即时生成的浮点数向量。
- 以最基础的 **线性回归（Linear Regression）** 为例：
 - 维度设定：问题维度 $d = 20$ ，上下文样本数 $k = 40$ 。
 - 生成逻辑：每次迭代前，随机采样一个真实的参数 $w \sim \mathcal{N}(0, I_d)$ 。
 - 采样输入：生成 $k + 1$ 个输入样本（包含测试样本） $x_i \sim \mathcal{N}(0, I_d)$ 。

- 计算标签: $y_i = w^T x_i$ 。
- 拼接成 Prompt: 最终喂给模型的序列 P 是一个交替的序列 $(x_1, y_1, x_2, y_2, \dots, x_k, y_k, x_{test}, y_{dummy})$ 。
- 线性回归可以进一步改成非线性回归 (比如 ReLU 与 Linear 叠加), 或者更复杂的物理系统。

把 Prompt 映射到高维 Embedding 空间 双通道映射与“拉链式”交织

由于 x 和 y 代表不同的物理量 (一个是空间坐标/特征, 一个是目标观测值), 我们需要赋予它们独立的投影空间:

1. 定义两个独立的线性层:

```
read_in_x = nn.Linear(20, 256)
read_in_y = nn.Linear(1, 256)
```

2. 把前 k 个 x_i 丢给 `read_in_x`, 变成张量 X_{emb} , 形状为 $[batch, k, 256]$ 。
3. 把前 k 个 y_i 丢给 `read_in_y`, 变成张量 Y_{emb} , 形状为 $[batch, k, 256]$ 。
4. 像拉拉链一样, 在 `seq_len` 维度上把 X_{emb} 、 Y_{emb} 交织 (Interleave) 起来, 拼接成最终长度为 $2k$ 的提示向量 P 。

在工程实现中, 我们在 x_{test} 后填充了一个 y_{dummy} (如 0), 使实际拼接长度变为 $2k+2$ (偶数)。因此, 真正对 x_{test} 的预测结果位于输出序列的倒数第二个位置, 即索引 $[-2]$ 处。

1.2 代码

```
[3]: import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
import time
from operator import sub
import copy
import numpy as np
import math
import concurrent.futures
import threading
```

1.2.1 位置编码 (PE, Position Encoding)

绝对位置编码 (APE, Absolute Position Encoding)

```
[4]: class APE(nn.Module):
    def __init__(self, d_model, max_seq_len=4096, b=10000):
        super().__init__()
        theta_i = 1 / (b ** (torch.arange(0, d_model, 2).float() / d_model)) #
        ↪ [d_model/2]
        m = torch.arange(max_seq_len).float() # [max_seq_len]
        m_theta_i = torch.outer(m, theta_i) # [max_seq_len, d_model/2]
        # APE 核心: 直接生成一个完整的 pe 矩阵, 偶数维度放 sin, 奇数维度放 cos
        stacked=torch.stack((torch.sin(m_theta_i), torch.cos(m_theta_i)),
        ↪ dim=-1) # [max_seq_len, d_model/2, 2]
        pe=stacked.flatten(1,2) # [max_seq_len, d_model]
        self.register_buffer('pe', pe.unsqueeze(0)) # shape: [1, max_seq_len,
        ↪ d_model]

    def forward(self, x):
        # x shape: [batch_size, seq_len, d_model]
        return x + self.pe[:, :x.shape[1], :]
```

Learned APE

```
[5]: class LearnedAPE(nn.Module):
    def __init__(self, d_model, max_seq_len=4096):
        super().__init__()
        self.pe = nn.Embedding(max_seq_len, d_model)

    def forward(self, x):
        # x shape: [batch_size, seq_len, d_model]
        seq_len = x.shape[1]
        positions = torch.arange(seq_len, device=x.device)
        return x + self.pe(positions).unsqueeze(0)
```

ALiBi (Attention with Linear Biases) ALiBi 通过更改内积的方式引入线性位置偏置, 从而实现了位置编码的功能。

$$\text{Score}(q_i, k_j) = q_i \cdot k_j - m \cdot (i - j) \quad (i \geq j)$$

其中 m 是一个与头数 (H) 相关的斜率参数, 硬编码为

$$m_h = 2^{-\frac{sh}{H}} \quad h = 0, 1, \dots, H - 1$$

```
[6]: class ALiBi(nn.Module):
    def __init__(self, num_heads, max_seq_len=4096):
        super().__init__()
        dist = torch.arange(max_seq_len) # [max_seq_len]
        dist_matrix = torch.abs(dist.view(-1, 1) - dist.view(1, -1)) #
        ↪ [max_seq_len, max_seq_len]
        bias = -(2**-(8*torch.arange(1,num_heads+1) / num_heads)).view(-1, 1,
        ↪ 1) * dist_matrix.unsqueeze(0) # [num_heads, max_seq_len, max_seq_len]
        # 因果掩码
        tril = torch.tril(torch.ones(max_seq_len, max_seq_len)).bool() # torch.
        ↪ tril (triangular lower) 提取下三角元素, 并把上三角元素置 0
        mask = torch.zeros(max_seq_len, max_seq_len).masked_fill(~tril,
        ↪ float('-inf')) # 下三角元素置 0, 上三角元素置 -inf
        self.register_buffer('fused_mask', bias + mask) # [num_heads,
        ↪ max_seq_len, max_seq_len]

    def forward(self, seq_len):
        # score shape: [batch_size, num_heads, seq_len, seq_len]
        return self.fused_mask[:, :seq_len, :seq_len]
```

原来的 RoPE (Rotary Position Encoding)

```
[7]: class RoPE(nn.Module):
    def __init__(self, d_k, max_seq_len = 4096, b=10000):
        super().__init__()
        theta_i = 1/(b**(torch.arange(0,d_k,2).float()/d_k)) # \theta_i =
        ↪ b^{-\frac{2i}{d}}, \quad i \in \{0,1,\dots,\frac{d}{2}-1\}
        m = torch.arange(max_seq_len).float() # m = [0,1,...,seq_len-1]
        m_theta_i = torch.outer(m, theta_i) # [seq_len, d_k/2]
        cos = torch.cos(torch.cat((m_theta_i, m_theta_i), dim=-1)) # [seq_len,
        ↪ d_k]
        sin = torch.sin(torch.cat((m_theta_i, m_theta_i), dim=-1)) # [seq_len,
        ↪ d_k]
        self.register_buffer('cos', cos[None, None, :, :]) # 好处: cos 和 sin
        不需要更新参数, 注册为 buffer 后会自动放到正确的设备上
        self.register_buffer('sin', sin[None, None, :, :]) # 扩充维度以适应后续
        计算
```

```

def forward(self, x): # 适用于 Q、K (V 不需要位置编码!)
    # x: [batch_size, num_heads, seq_len, d_k]
    seq_len = x.shape[2]
    d_2 = x.shape[-1] // 2
    cos = self.cos[:, :, :seq_len, :] # [1, 1, seq_len, d_k]
    sin = self.sin[:, :, :seq_len, :]
    x_first_half = x[..., :d_2]
    x_second_half = x[..., d_2:]
    x_flip = torch.cat((-x_second_half, x_first_half), dim=-1)
    x_out = x * cos + x_flip * sin # 旋转位置编码
    return x_out

```

自创: Multi-Scale Untied Positional Encoding (MS-UPE)

```

[8]: class MS_UPE(nn.Module):
    def __init__(self, num_heads, d_k, max_seq_len = 4096, b_0=10000,
        ↪ head_ratio=2):
        super().__init__()
        b=(b_0*(head_ratio*(torch.arange(0, num_heads))))).float() # [num_heads]
        # \theta_i = b^{-\frac{2i}{d}}, \quad i \in \{0, 1, \dots, \frac{d}{2}-1\}
        theta_i = 1/(b.unsqueeze(1)*(torch.arange(0, d_k, 2).float()/d_k).
        ↪ unsqueeze(0)) # [num_heads, d_k/2]
        m = torch.arange(max_seq_len).float() # m = [0, 1, ..., seq_len-1]
        m_theta_i = torch.einsum('m,hk->hmk', m, theta_i) # [num_heads,
        ↪ seq_len, d_k/2]
        pe = torch.cat((torch.cos(m_theta_i), torch.sin(m_theta_i)), dim=-1) #
        ↪ [num_heads, seq_len, d_k]
        self.register_buffer('pe', pe[None, :, :, :]) # 好处: pe 不需要更新参数,
        注册为 buffer 后会自动放到正确的设备上

    def forward(self, x): # 适用于 Q、K (V 不需要位置编码!)
        # x: [batch_size, num_heads, seq_len, d_k]
        seq_len = x.shape[2]
        pe = self.pe[:, :, :seq_len, :] # [1, num_heads, seq_len, d_k]
        x_out = x + pe
        return x_out

```

1.2.2 Multi-Head Attention

注：APE 应该加在 ToyModel 上而不是 TransformerBlock 上，因为我们只需要最开头输入处编码位置，而不是叠加 num_blocks 次。

```
[9]: class MultiHeadAttention(nn.Module):
    def __init__(self, num_heads, d_model, max_seq_len=4096,
        ↪pe_type='learned_ape', b_rope_or_upe=10000, head_ratio_upe=2):
        super().__init__()
        assert d_model % num_heads == 0, "d_model must be divisible by
        ↪num_heads"
        # 维度属性
        self.num_heads = num_heads
        self.d_model = d_model
        self.d_k = d_model // num_heads
        # 权重矩阵
        self.W_Q = nn.Linear(d_model, d_model, bias=False)
        self.W_K = nn.Linear(d_model, d_model, bias=False)
        self.W_V = nn.Linear(d_model, d_model, bias=False)
        self.W_O = nn.Linear(d_model, d_model, bias=False)
        # 位置编码模块
        self.pe_type = [pe_type] if isinstance(pe_type, str) else pe_type
        self.qk_pe_modules = nn.ModuleList()
        self.sc_pe_modules = nn.ModuleList()
        for pe in self.pe_type:
            if pe == 'rope':
                pe_module = RoPE(self.d_k, max_seq_len, b=b_rope_or_upe)
                self.qk_pe_modules.append(pe_module) # RoPE 应用于 Q 和 K
            elif pe == 'ms_upe':
                pe_module = MS_UPE(num_heads, self.d_k, max_seq_len,
        ↪b_0=b_rope_or_upe, head_ratio=head_ratio_upe)
                self.qk_pe_modules.append(pe_module) # MS-UPE 应用于 Q 和 K
            elif pe == 'alibi':
                pe_module = ALiBi(num_heads, max_seq_len)
                self.sc_pe_modules.append(pe_module) # ALiBi 应用于 score 矩阵
            elif pe in ['ape', 'learned_ape']:
                pass # APE 和 Learned APE 在输入时直接加到 x 上，不需要单独模块
```

```

# 因果掩码
tril = torch.tril(torch.ones(max_seq_len, max_seq_len)).bool() # torch.
↪tril (triangular lower) 提取下三角元素, 并把上三角元素置 0
mask = torch.zeros(max_seq_len, max_seq_len).masked_fill(~tril,
↪float('-inf')) # 下三角元素置 0, 上三角元素置 -inf
self.register_buffer('mask', mask[None, None, :, :]) # [1, 1, seq_len,
↪seq_len]

def forward(self, x): # x: [batch_size, seq_len, d_model]
    # 线性投影与分头
    batch_size, seq_len, _ = x.shape
    Q = self.W_Q(x).view(batch_size, seq_len, self.num_heads, self.d_k).
↪transpose(1, 2) # [batch_size, num_heads, seq_len, d_k]
    K = self.W_K(x).view(batch_size, seq_len, self.num_heads, self.d_k).
↪transpose(1, 2) # 交换 (1, 2) 是因为之后 softmax 时要进行矩阵乘法 (只乘后两维)
    V = self.W_V(x).view(batch_size, seq_len, self.num_heads, self.d_k).
↪transpose(1, 2) # 在 seq_len 维度上分配注意力
    # 根据选择的 PE 类型对 Q 和 K 进行位置编码
    for pe_module in self.qk_pe_modules:
        Q = pe_module(Q)
        K = pe_module(K)
    if not self.training:
        # 缩放点积注意力与因果掩码 Causal Masking
        ## Query-Key 点积计算注意力得分
        scores = torch.matmul(Q, K.transpose(-2, -1)) / (self.d_k ** 0.5)
↪# [batch_size, num_heads, seq_len, seq_len]
        if self.sc_pe_modules:
            fused_mask = self.sc_pe_modules[0](seq_len) # [num_heads,
↪seq_len, seq_len]
            scores = scores + fused_mask # 将 ALiBi 的偏置加到得分上
            attention = F.softmax(scores, dim=-1) # [batch_size,
↪num_heads, seq_len, seq_len]
        else:
            ## 因果掩码和 softmax 计算注意力权重

```

```

        attention = F.softmax(scores + self.mask[..., :seq_len, :
↪seq_len], dim=-1) # [batch_size, num_heads, seq_len, seq_len]
        self.captured_attention = attention # ** 新增 **: 捕获注意力矩阵, 用
hook 记录下来, 方便后续分析
        ## 乘以 Value 完成加权求和
        out = torch.matmul(attention, V) # [batch_size, num_heads, ↪
↪seq_len, d_k]
    else:
        if self.sc_pe_modules:
            fused_mask = self.sc_pe_modules[0](seq_len) # [num_heads, ↪
↪seq_len, seq_len]
            out = F.scaled_dot_product_attention(Q, K, V, ↪
↪attn_mask=fused_mask, is_causal=False) # [batch_size, num_heads, seq_len, ↪
↪d_k]
        else:
            out = F.scaled_dot_product_attention(Q, K, V, is_causal=True) ↪
↪# [batch_size, num_heads, seq_len, d_k]
            # 多头拼接并乘以输出矩阵 (必须先内存连续化 (contiguous) 再 view, 否则会报
            错)
            out = out.transpose(1, 2).contiguous().view(batch_size, seq_len, -1) # ↪
↪[batch_size, seq_len, d_model]
            H = self.W_O(out) # [batch_size, seq_len, d_model]
        return H

```

1.2.3 SwiGLU 和单个 Transformer Block 架构不变

```

[10]: class SwiGLU(nn.Module):
    def __init__(self, d_model, d_hidden):
        super().__init__()
        self.W = nn.Linear(d_model, d_hidden, bias=False)
        self.V = nn.Linear(d_model, d_hidden, bias=False)
        self.W2 = nn.Linear(d_hidden, d_model, bias=False)
    def forward(self, x):
        x1 = F.silu(self.W(x))
        x2 = self.V(x)
        x_out = self.W2(x1 * x2)

```



```
return x_out
```

```
[11]: class TransformerBlock(nn.Module):
    def __init__(self, num_heads, d_model, max_seq_len=4096, multiplier=4,
        norm_type='layernorm', ffn_type='gelu', pe_type='learned_ape',
        b_rope_or_upe=10000, head_ratio_upe=2):
        super().__init__()
        if norm_type == 'rmsnorm':
            self.norm1 = nn.RMSNorm(d_model) # 第一次归一化
            self.norm2 = nn.RMSNorm(d_model) # 第二次归一化
        elif norm_type == 'layernorm':
            self.norm1 = nn.LayerNorm(d_model) # 第一次归一化
            self.norm2 = nn.LayerNorm(d_model) # 第二次归一化
        self.attention = MultiHeadAttention(num_heads, d_model,
        max_seq_len=max_seq_len, pe_type=pe_type, b_rope_or_upe=b_rope_or_upe,
        head_ratio_upe=head_ratio_upe)
        if ffn_type == 'swiglu':
            self.ffn = SwiGLU(d_model, d_model * multiplier)
        elif ffn_type == 'gelu':
            self.ffn = nn.Sequential(
                nn.Linear(d_model, d_model * multiplier),
                nn.GELU(),
                nn.Linear(d_model * multiplier, d_model)
            )

    def forward(self, x):
        # 第一次归一化 Pre-Norm
        x_norm1 = self.norm1(x)
        # 全局信息交互 Multi-Head Attention (MHA)
        h = self.attention(x_norm1)
        # 第一次残差连接
        x1 = x + h
        # 第二次归一化
        x_norm2 = self.norm2(x1)
        # 前馈网络 Feed-Forward Network (FFN)
        x_out = self.ffn(x_norm2)
```

```

# 第二次残差连接
x2 = x1 + x_out
return x2

```

1.2.4 保留 Attention Probe

```

[12]: class AttentionProbe:
    def __init__(self):
        self.captured_data = []
    def __call__(self, module, input, output):
        if getattr(module, 'captured_attention', None) is not None:
            attention = module.captured_attention.detach().cpu().numpy()
            self.captured_data.append(attention)
        else:
            self.captured_data.append(None) # 如果没有捕获到注意力矩阵, 记录一个
None 占位符
    def reset(self):
        self.captured_data = []

```

1.2.5 Toy Model: Looped Transformer

```

[13]: class ToyModel(nn.Module):
    def __init__(self, num_blocks, num_heads, d_model, max_seq_len=4096,
        norm_type='layernorm', ffn_type='gelu', pe_type='learned_ape',
        ↪b_rope_or_upe=10000, head_ratio_upe=2,
        loop=True, residual_gate=(1,1), residual_gate_type='fixed',
        ↪residual_random=(1,0.1)):
        super().__init__()
        self.loop = loop
        if loop:
            self.transformer_block = TransformerBlock(num_heads, d_model,
            ↪max_seq_len=max_seq_len, norm_type=norm_type, ffn_type=ffn_type,
            ↪pe_type=pe_type, b_rope_or_upe=b_rope_or_upe, head_ratio_upe=head_ratio_upe)
            ↪# 这里保证了各层权重始终相同
            self.probe = AttentionProbe()
            self.transformer_block.attention.register_forward_hook(self.probe)
            self.captured_attention = None

```

```

else:
    self.transformer_block = nn.ModuleList([
        TransformerBlock(num_heads, d_model, max_seq_len=max_seq_len,
        ↪norm_type=norm_type, ffn_type=ffn_type, pe_type=pe_type,
        ↪b_rope_or_upe=b_rope_or_upe, head_ratio_upe=head_ratio_upe)
        for _ in range(num_blocks)
    ])
    self.probe = [AttentionProbe() for _ in range(num_blocks)]
    for block, probe in zip(self.transformer_block, self.probe):
        block.attention.register_forward_hook(probe)

    residual_random = torch.tensor(residual_random, dtype=torch.float32)
    if residual_gate == 'random':
        residual_gate_tensor = torch.
        ↪randn(2)*residual_random[1]+residual_random[0] # 随机初始化门控参数，初始正态分
        布
    else:
        residual_gate_tensor = torch.tensor(residual_gate, dtype=torch.
        ↪float32) # 将输入的元组转换为张量，方便后续计算
        if residual_gate_type == 'fixed':
            self.register_buffer('residual_gate', residual_gate_tensor) # 固定
            的门控参数，注册为 buffer 使其成为模型状态的一部分，但不参与梯度更新
        elif residual_gate_type == 'learnable_scalar':
            self.residual_gate = nn.Parameter(torch.
            ↪ones(2)*residual_gate_tensor) # 可学习的标量门控参数，初始值为 1.0
        elif residual_gate_type == 'learnable_vector':
            if residual_gate == 'random':
                self.residual_gate = nn.Parameter(torch.randn(d_model,
                ↪2)*residual_random[1]+residual_random[0]) # 可学习的向量门控参数，初始值为随机
                向量
            else:
                self.residual_gate = nn.Parameter(torch.ones(d_model,
                ↪2)*residual_gate_tensor) # 可学习的向量门控参数，初始值为全 1 向量

    pe_type = self.transformer_block.attention.pe_type
    self.ape = nn.ModuleList()

```

```

for pe in pe_type:
    if pe == 'ape':
        self.ape.append(APE(d_model, max_seq_len=max_seq_len))
    elif pe == 'learned_ape':
        self.ape.append(LearnedAPE(d_model, max_seq_len=max_seq_len))
    elif pe in ['rope', 'ms_upe']:
        pass # RoPE 和 MS_UPE 在 MultiHeadAttention 内部处理位置编码，不需要单独的模块在这里处理

```

```

self.num_blocks = num_blocks
self.num_heads = num_heads
self.d_model = d_model
self.max_seq_len = max_seq_len
self.pe_type = pe_type

```

```

def forward(self, x_0, num_eff=15, current_blocks=None):
    # num_eff 是需要加入误差计算的有效层数，也即理论公式中的  $T$ 
    # x 的形状为 [batch_size, seq_len, d_model]
    a=self.residual_gate[...,0]
    b=self.residual_gate[...,1]
    for ape_module in self.ape:
        x_0 = ape_module(x_0) # 绝对位置编码只加在最开始输入处
    x = x_0
    if current_blocks is None:
        current_blocks = self.num_blocks
    b_0 = max(0, current_blocks - num_eff) # 计算需要加入误差计算的有效层数对应的起始层索引
    outputs = []
    if self.loop:
        self.probe.reset() # 每次前向传播前重置捕获的数据，避免混淆不同次前向传播的注意力矩阵
        for i in range(current_blocks):
            if i == b_0:
                x= x.detach() # 释放前面的计算图
                x.requires_grad_(True)
            x = self.transformer_block(a*x+b*x_0)

```

```

        if i >= b_0:
            outputs.append(x)
            self.captured_attention = list(self.probe.captured_data) # 显式 copy
一份数据，避免后续被修改
        else:
            for probe in self.probe:
                probe.reset()
            for i, block in enumerate(self.transformer_block):
                if i == b_0:
                    x = x.detach() # 释放前面的计算图
                    x.requires_grad_(True)
                    x = block(a*x+b*x_0) # x: [batch_size, seq_len, d_model]
                    if i >= b_0:
                        outputs.append(x)
                        self.captured_attention = [probe.captured_data for probe in self.
→probe]
                outputs = torch.stack(outputs, dim=1) # [batch_size, num_eff, seq_len,
→d_model]
            return outputs

```

1.2.6 Downstream Task: Linear Regression

转换头

```

[14]: class RegressionHead(nn.Module):
    def __init__(self, d_model, d_x, d_y=1, bias=False, init_scale=None):
        super().__init__()
        self.read_in_x = nn.Linear(d_x, d_model, bias=bias)
        self.read_in_y = nn.Linear(d_y, d_model, bias=bias)
        if init_scale is not None:
            nn.init.normal_(self.read_in_x.weight, mean=0.0, std=init_scale)
            nn.init.normal_(self.read_in_y.weight, mean=0.0, std=init_scale)
    def forward(self, x_data, y_data):
        # k 相当于 seq_len 的一半，因为 x 和 y 交织在一起，所以总的 seq_len 是 2k
        # x_data: [batch_size, k, d_x]
        # y_data: [batch_size, k, d_y]
        x_emb = self.read_in_x(x_data) # [batch_size, k, d_model]
        y_emb = self.read_in_y(y_data) # [batch_size, k, d_model]

```

```

# 交织成 (x_1, y_1, x_2, y_2, ..., x_k, y_k)
stacked = torch.stack((x_emb, y_emb), dim=2) # [batch_size, k, 2, d_model]
Prompt = stacked.flatten(1,2) # [batch_size, 2k, d_model]
return Prompt

```

损失函数

```

[15]: class PredictionLoss(nn.Module):
    def __init__(self, d_model, d_y=1, loss_type='mse', layer_weight_decay=1.0, seq_weight_decay=1.0):
        super().__init__()
        self.read_out = nn.Linear(d_model, d_y)
        self.layer_weight_decay = layer_weight_decay
        self.seq_weight_decay = seq_weight_decay
        if loss_type == 'mse':
            self.loss_fn = nn.MSELoss(reduction='none')
        elif loss_type == 'l1':
            self.loss_fn = nn.L1Loss(reduction='none')

    def forward(self, outputs, y_true, is_eval=False, sink_padding=None):
        # outputs: [batch_size, num_eff, seq_len, d_model]
        # y_true: [batch_size, seq_len/2, 1]
        if is_eval:
            y_outputs = outputs[:, -1, -2, :] # 只取最后一层的最后一个位置的输出
            # 作为预测值
            y_pred_final = self.read_out(y_outputs) # [batch_size, d_y]
            return y_pred_final
            y_outputs = outputs[:, :, 0::2, :] # [batch_size, num_eff, seq_len/2, d_model]
            y_preds = self.read_out(y_outputs) # [batch_size, num_eff, seq_len/2, d_y]
            if sink_padding is not None:
                y_preds = y_preds[:, :, sink_padding:, :]
                y_true = y_true[:, sink_padding:, :]
            y_pred_norm = torch.sqrt(torch.mean(y_preds.detach() ** 2)).item()
            y_true_norm = torch.sqrt(torch.mean(y_true.detach() ** 2)).item()

```

```

        loss_unreduced = self.loss_fn(y_preds, y_true.unsqueeze(1).
    ↪ expand_as(y_preds)) # [batch_size, num_eff, seq_len/2, d_y]
        if self.layer_weight_decay != 1.0:
            num_eff = outputs.shape[1]
            layer_weights = self.layer_weight_decay ** torch.arange(num_eff-1,
    ↪ -1, -1, device=outputs.device) # 从最后一层到第一层递减的权重
            layer_weights = layer_weights / layer_weights.mean()
            loss_unreduced = loss_unreduced * layer_weights.view(1, num_eff, 1,
    ↪ 1) # 按层加权
        if self.seq_weight_decay != 1.0:
            k=loss_unreduced.shape[2]
            seq_weights = self.seq_weight_decay ** torch.arange(k-1, -1, -1,
    ↪ device=outputs.device) # 从最后一次预测到第一次预测递减的权重
            seq_weights = seq_weights / seq_weights.mean()
            loss_unreduced = loss_unreduced * seq_weights.view(1, 1, k, 1) # 按
位置加权
        return loss_unreduced.mean(), y_pred_norm, y_true_norm

```

拼装

```

[16]: class RegressionSolver(nn.Module):
    def __init__(self, num_blocks, num_heads, d_model, d_x, d_y, max_seq_len,
        norm_type='layernorm', ffn_type='gelu', pe_type='learned_ape',
    ↪ b_rope_or_upe=10000, head_ratio_upe=2,
        loop=True, loss_type='mse',
        residual_gate=(1,1), residual_gate_type='fixed',
    ↪ residual_random=(1,0.1),
        bias=False, init_scale=None,
        layer_weight_decay=1.0, seq_weight_decay=1.0):
        super().__init__()
        self.toy_model = ToyModel(num_blocks, num_heads, d_model, max_seq_len,
            norm_type=norm_type, ffn_type=ffn_type,
    ↪ pe_type=pe_type,
            loop=loop, b_rope_or_upe=b_rope_or_upe,
    ↪ head_ratio_upe=head_ratio_upe,
            residual_gate=residual_gate,
    ↪ residual_gate_type=residual_gate_type, residual_random=residual_random)

```

```

        self.head = RegressionHead(d_model, d_x, d_y, bias=bias,
↪init_scale=init_scale)

        self.loss_fn = PredictionLoss(d_model, d_y, loss_type=loss_type,
↪layer_weight_decay=layer_weight_decay, seq_weight_decay=seq_weight_decay)

    def forward(self, x_data, y_data, num_eff, current_blocks=None,
↪is_eval=False, sink_padding=None):
        Prompt = self.head(x_data, y_data) # [batch_size, seq_len, d_model]
        outputs = self.toy_model(Prompt, num_eff,
↪current_blocks=current_blocks) # [batch_size, num_eff, seq_len, d_model]
        return self.loss_fn(outputs, y_data, is_eval=is_eval,
↪sink_padding=sink_padding)

```

1.2.7 数据生成与加载

数据生成

线性回归数据

```

[17]: def linear_data_generator(batch_size, seq_len, d_x, d_y, device='cpu',
↪generator=None):
    w = torch.randn(batch_size, d_x, d_y, device=device, generator=generator)/
↪(d_x ** 0.5) # 真实线性函数 w: [batch_size, d_x, d_y]
    x_data = torch.randn(batch_size, seq_len//2, d_x, device=device,
↪generator=generator) # 输入特征 x: [batch_size, seq_len//2, d_x]
    y_data = x_data @ w # 计算标签 y = x @ w: [batch_size, seq_len//2, d_y]
    return x_data, y_data

```

非线性回归数据

```

[ ]: def nonlinear_data_generator(batch_size, seq_len, d_x, d_y, d_hidden,
↪function_callable, device='cpu', generator=None, nonlinear_funcs=['relu']):
    '''
        linear(d_x->d_hidden) + nonlinear_func + linear(d_hidden->d_y) 的形式生成数
        据
        function_callable: 非线性函数, 如 lambda x: F.relu(2*torch.sin(x))+torch.
↪cos(x)
        d_hidden: 隐藏层维度, 控制非线性函数的输入输出维度。
    '''

```


若 $d_{hidden}=d_x$ ，则第一层线性变换默认设定为恒等映射，数据生成过程变为 $nonlinear_func + linear$ 的形式；

若 $d_{hidden}=d_y$ ，则第二层线性变换默认设定为恒等映射，数据生成过程变为 $linear + nonlinear_func$ 的形式。

```
'''
    w1 = torch.randn(batch_size, d_x, d_hidden, device=device,
    ↪generator=generator)/(d_x ** 0.5) # 第一层权重 w1: [batch_size, d_x,
    ↪d_hidden]
    w2 = torch.randn(batch_size, d_hidden, d_y, device=device,
    ↪generator=generator)/(d_hidden ** 0.5) # 第二层权重 w2: [batch_size,
    ↪d_hidden, d_y]
    x_data = torch.randn(batch_size, seq_len//2, d_x, device=device,
    ↪generator=generator) # 输入特征 x: [batch_size, seq_len//2, d_x]
    if d_x==d_hidden:
        w1=torch.eye(d_x, device=device).unsqueeze(0).expand(batch_size, -1,
    ↪-1) # 如果输入维度和隐藏层维度相同，变为 nonlinear_func + linear 的形式
    if d_hidden==d_y:
        w2=torch.eye(d_hidden, device=device).unsqueeze(0).expand(batch_size,
    ↪-1, -1) # 如果隐藏层维度和输出维度相同，变为 linear + nonlinear_func 的形式
    hidden = x_data @ w1 # [batch_size, seq_len//2, d_hidden]
    # 应用非线性函数
    hidden = function_callable(hidden) # [batch_size, seq_len//2, d_hidden]
    # 第二层线性变换
    y_data = hidden @ w2 # [batch_size, seq_len//2, d_y]
    return x_data, y_data
```

数据加载器

```
[19]: def dataloader(batch_size=64, seq_len=80, d_x=20, d_y=1, device='cpu',
    ↪data_type='linear', d_hidden=None, function_callable=None, sink_padding=1,
    ↪generator=None):
    while True:
        if data_type == 'linear':
            x_data, y_data = linear_data_generator(batch_size, seq_len, d_x,
    ↪d_y, device=device, generator=generator)
            elif data_type == 'nonlinear':
```

```

        x_data, y_data = nonlinear_data_generator(batch_size, seq_len, d_x,
↪d_y, d_hidden, function_callable, device=device, generator=generator)
        if sink_padding is not None:
            x_data = F.pad(x_data, (0, 0, sink_padding, 0), value=0) # 在
seq_len 维度最前面添加 sink_padding 个全 0 向量
            y_data = F.pad(y_data, (0, 0, sink_padding, 0), value=0)
        yield x_data, y_data

```

1.2.8 实验模块

```

[20]: class LoopedTransformerExperiment:
    def __init__(self, num_blocks, num_heads=8, d_model=256, lr=1e-4,
↪gate_lr_ratio=100, max_seq_len=4096, d_x=20, d_y=1,
        seed=None, experiment_name='Experiment',
        norm_type='layernorm', ffn_type='gelu', pe_type='learned_ape',
↪b_rope_or_upe=10000, head_ratio_upe=2,
        optimizer_type='adam', wd_adamw=0.01,
        loop=True, loss_type='mse', bias=False, init_scale=None,
        residual_gate=(1,1), residual_gate_type='fixed',
↪residual_random=(1,0.1),
        task='regression', timing=True,
        layer_weight_decay=1.0, seq_weight_decay=1.0):
        if timing:
            start_time = time.time()
            # 设备选择: 优先使用 MPS (适用于 Apple Silicon), 其次是 CUDA (适用于
↪NVIDIA GPU), 最后退回 CPU
            if torch.backends.mps.is_available():
                self.device = torch.device("mps")
            elif torch.cuda.is_available():
                self.device = torch.device("cuda")
            else:
                self.device = torch.device("cpu")
            self.is_offloaded = False
            self.generator = torch.Generator(device=self.device)
            if seed is not None:
                torch.manual_seed(seed)

```

```

        self.generator.manual_seed(seed)
# 模型和优化器
self.task = task
if task == 'regression':
    self.model = RegressionSolver(num_blocks, num_heads, d_model,
↪d_x=d_x, d_y=d_y, max_seq_len=max_seq_len,
                                norm_type=norm_type,
↪ffn_type=ffn_type, pe_type=pe_type, b_rope_or_upe=b_rope_or_upe,
                                head_ratio_upe=head_ratio_upe,
↪loop=loop, loss_type=loss_type,
                                bias=bias, init_scale=init_scale,
                                residual_gate=residual_gate,
↪residual_gate_type=residual_gate_type, residual_random=residual_random,
                                )
↪layer_weight_decay=layer_weight_decay, seq_weight_decay=seq_weight_decay
                                ).to(self.device)

    gate_params=[]
    base_params=[]
    for name, param in self.model.named_parameters():
        if 'residual_gate' in name:
            gate_params.append(param)
        else:
            base_params.append(param)
    param_groups = [{'params': base_params},
                    {'params': gate_params, 'lr': lr * gate_lr_ratio}]
    if optimizer_type == 'adam':
        self.optimizer = torch.optim.Adam(param_groups, lr=lr)
    elif optimizer_type == 'sgd':
        self.optimizer = torch.optim.SGD(param_groups, lr=lr)
    elif optimizer_type == 'adamw':
        self.optimizer = torch.optim.AdamW(param_groups, lr=lr,
↪weight_decay=wd_adamw)

    self.d_x = d_x
    self.d_y = d_y
    self.num_blocks = num_blocks
    self.init_time = None

```

```

        if experiment_name is not None:
            if timing:
                self.init_time = time.time() - start_time
                print(f"{experiment_name} initialized in {self.init_time:.2f}␣
↪seconds. Using device: {self.device}")
            else:
                print(f"{experiment_name} initialized. Using device: {self.
↪device}")

    def train(self, batch_size=64, seq_len=80, epochs=20, data_type='linear',
              d_hidden=None, function_callable=None,
              num_eff=15, sink_padding=None, scheduled_training=True,␣
↪scheduler_type=None, lr_scale=0.1, step_size_scheduler=10,
              print_every=None, timing=False):
        # x_data: [batch_size, k, d_x]
        # y_data: [batch_size, k, d_y]
        self.model.train()
        self.max_torch_vram = 0
        self.max_metal_vram = 0
        self.train_time = None
        self.data_loader = dataloader(batch_size, seq_len, self.d_x, self.d_y,␣
↪device=self.device, data_type=data_type, sink_padding=sink_padding,␣
↪generator=self.generator, d_hidden=d_hidden,␣
↪function_callable=function_callable)
        self.loss_history = []
        self.y_pred_norm_history = []
        self.y_true_norm_history = []
        self.residual_gate_history = []
        if scheduler_type == 'cosine':
            scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(self.
↪optimizer, T_max=epochs, eta_min=lr_scale*self.optimizer.
↪param_groups[0]['lr'])
        elif scheduler_type == 'step':
            scheduler = torch.optim.lr_scheduler.StepLR(self.optimizer,␣
↪step_size=step_size_scheduler, gamma=lr_scale)
        num_eff = min(num_eff, self.num_blocks) # 确保 num_eff 不超过总层数

```

```

    if timing:
        start_time = time.time()
    for epoch in range(epochs):
        self.optimizer.zero_grad()
        x_data, y_data = next(self.data_loader) # 从数据加载器中获取一个批次的
        数据

        if scheduled_training:
            current_blocks = min(num_eff + epoch, self.num_blocks) # 随着训练的
            进行, 逐渐增加参与误差计算的有效层数
        else:
            current_blocks = self.num_blocks

            loss, y_pred_norm, y_true_norm = self.model(x_data, y_data,
            ↪num_eff=num_eff, current_blocks=current_blocks, is_eval=False,
            ↪sink_padding=sink_padding) # 前向传播计算损失 [batch_size, num_eff, seq_len/
            ↪2, d_y]

            loss = loss.mean() # 对所有维度求平均得到一个标量损失值
            loss.backward() # 反向传播计算梯度

            if torch.backends.mps.is_available():
                self.max_metal_vram = max(self.max_metal_vram, torch.mps.
                ↪driver_allocated_memory() / (1024 ** 3))

                self.max_torch_vram = max(self.max_torch_vram, torch.mps.
                ↪current_allocated_memory() / (1024 ** 3))

            self.optimizer.step() # 更新模型参数
            if torch.backends.mps.is_available():
                torch.mps.empty_cache() # 清空 MPS 缓存, 释放未使用的显存
            elif torch.cuda.is_available():
                torch.cuda.empty_cache() # 清空 CUDA 缓存, 释放未使用的显存
            if scheduler_type is not None:
                scheduler.step() # 更新学习率

            self.y_pred_norm_history.append(y_pred_norm)
            self.y_true_norm_history.append(y_true_norm)
            self.loss_history.append(loss.item()) # 记录损失值
            self.residual_gate_history.append(self.model.toy_model.
            ↪residual_gate.detach().cpu().numpy() if hasattr(self.model.toy_model,
            ↪'residual_gate') else None) # 记录门控参数值

            if print_every is not None and (epoch + 1) % print_every == 0:

```

```

        print(f"Epoch {epoch+1}/{epochs}, Loss: {loss.item():.4f}, Norm_
↪Ratio: {y_pred_norm/y_true_norm if y_true_norm != 0 else None:.4f}, Current_
↪Blocks: {current_blocks}")

    if timing:
        self.train_time = time.time()-start_time
        print(f"Training completed in {self.train_time:.2f} seconds.")
        self.residual_gate_values = self.residual_gate_history[-1]
        self.final_loss = self.loss_history[-1]

    def evaluate(self, batch_size=64, seq_len=80, data_type='linear',
↪loss_type='mse', sink_padding=1):
        self.model.eval()
        with torch.no_grad():
            # 在评估模式下，直接使用所有层的输出进行预测（因为已经 no_grad 了，所以
            # 不需要防止梯度爆炸了）
            x_eval, y_eval = next(dataloader(batch_size, seq_len+1, self.d_x,
↪self.d_y, device=self.device, data_type=data_type,
↪sink_padding=sink_padding, generator=self.generator, d_hidden=None,
↪function_callable=None)) # 生成一个评估用的数据批次，注意这里的 seq_len 要比训练
            # 时大 1，因为我们需要预测最后一个位置的 y 值
            y_pred = self.model(x_eval, y_eval, num_eff=self.num_blocks,
↪current_blocks=self.num_blocks, is_eval=True, sink_padding=sink_padding) #
            ↪[batch_size, d_y]

            if loss_type == 'mse':
                mse=F.mse_loss(y_pred, y_eval[:, -1, :])
                print(f"MSELoss: {mse:.4f}") # 计算预测值与真实值之间的均方误差
                return mse.item()
            elif loss_type == 'l1':
                l1=F.l1_loss(y_pred, y_eval[:, -1, :])
                print(f"L1Loss: {l1:.4f}")
                return l1.item()

    def get_results(self, result_list:list[str]):
        all_results = {
            'init_time': self.init_time, #
            ↪float: 实验初始化时间（秒）

```

```

        'train_time': self.train_time,                                #_
↪float: 训练时间 (秒)

        'loss_history': self.loss_history,                            #_
↪list(float): 训练过程中每个 epoch 的损失值

        'y_pred_norm_history': self.y_pred_norm_history,             #_
↪list(float): 训练过程中每个 epoch 的预测值范数

        'y_true_norm_history': self.y_true_norm_history,             #_
↪list(float): 训练过程中每个 epoch 的真实值范数

        'y_norm_error_history': [pred - true for pred, true in zip(self.
↪y_pred_norm_history, self.y_true_norm_history)], # list(float): 训练过程中每个
epoch 的预测值范数与真实值范数之差的绝对值

        'y_norm_ratio_history': [pred / true if true != 0 else None for_
↪pred, true in zip(self.y_pred_norm_history, self.y_true_norm_history)], #_
↪list(float): 训练过程中每个 epoch 的预测值范数与真实值范数之比

        'final_loss': self.final_loss,                                #_
↪float: 最终的损失值

        'final_y_pred_norm': self.y_pred_norm_history[-1],           #_
↪float: 最终的预测值范数

        'final_y_true_norm': self.y_true_norm_history[-1],           #_
↪float: 最终的真实值范数

        'residual_gate_history': self.residual_gate_history,         #_
↪list[np.array[2] 或 np.array[d_model, 2]]: 训练过程中每个 epoch 的残差门控参数
值

        'final_residual_gate': self.residual_gate_values,            # np.
↪array[2] 或 np.array[d_model, 2]: 最终的残差门控参数值

        'captured_attention': self.model.toy_model.captured_attention #_
↪list[np.array[batch_size, num_heads, seq_len, seq_len]]: 捕获的注意力权重
    }

    if any("residual_gate" in key for key in result_list) and self.
↪residual_gate_values is not None:

        if len(self.residual_gate_values.shape) == 1:
            all_results['residual_gate_history_a'] = [gate[0] for gate in_
↪self.residual_gate_history]
            all_results['residual_gate_history_b'] = [gate[1] for gate in_
↪self.residual_gate_history]

```

```

        all_results['residual_gate_history_a_relative'] = [gate[0]-self.
↪residual_gate_history[0][0] for gate in self.residual_gate_history]
        all_results['residual_gate_history_b_relative'] = [gate[1]-self.
↪residual_gate_history[0][1] for gate in self.residual_gate_history]
        # 标量门控的 _mean 就是自身，便于与向量门控混合绘图
        all_results['residual_gate_history_a_mean'] =_
↪all_results['residual_gate_history_a']
        all_results['residual_gate_history_b_mean'] =_
↪all_results['residual_gate_history_b']
        all_results['residual_gate_history_a_mean_relative'] =_
↪all_results['residual_gate_history_a_relative']
        all_results['residual_gate_history_b_mean_relative'] =_
↪all_results['residual_gate_history_b_relative']
        all_results['final_residual_gate_a'] = self.
↪residual_gate_values[0]
        all_results['final_residual_gate_b'] = self.
↪residual_gate_values[1]
        all_results['final_residual_gate_a_mean'] =_
↪all_results['final_residual_gate_a']
        all_results['final_residual_gate_b_mean'] =_
↪all_results['final_residual_gate_b']
        elif len(self.residual_gate_values.shape) == 2:
            all_results['residual_gate_history_a_mean'] = [gate[:,0].mean()_
↪for gate in self.residual_gate_history]
            all_results['residual_gate_history_b_mean'] = [gate[:,1].mean()_
↪for gate in self.residual_gate_history]
            all_results['residual_gate_history_a_mean_relative'] = [gate[:
↪,0].mean()-self.residual_gate_history[0][:,0].mean() for gate in self.
↪residual_gate_history]
            all_results['residual_gate_history_b_mean_relative'] = [gate[:
↪,1].mean()-self.residual_gate_history[0][:,1].mean() for gate in self.
↪residual_gate_history]
            all_results['residual_gate_history_a_std'] = [gate[:,0].std()_
↪for gate in self.residual_gate_history]

```



```

        all_results['residual_gate_history_b_std'] = [gate[:,1].std()
↪for gate in self.residual_gate_history]
        all_results['final_residual_gate_a_mean'] = self.
↪residual_gate_values[:,0].mean()
        all_results['final_residual_gate_b_mean'] = self.
↪residual_gate_values[:,1].mean()
        all_results['final_residual_gate_a_std'] = self.
↪residual_gate_values[:,0].std()
        all_results['final_residual_gate_b_std'] = self.
↪residual_gate_values[:,1].std()
        invalid_keys = [key for key in result_list if key not in all_results]
        if invalid_keys:
            print(f"Warning: The following requested results are not available,
↪and will be ignored: {invalid_keys}")
        if result_list is not None:
            return {key: all_results[key] for key in result_list if key in
↪all_results}

def offload_to_cpu(self):
    self.model.to('cpu')
    for state in self.optimizer.state.values():
        for k, v in state.items():
            if isinstance(v, torch.Tensor):
                state[k] = v.cpu()
    self.is_offloaded = True

def load_to_device(self):
    self.model.to(self.device)
    for state in self.optimizer.state.values():
        for k, v in state.items():
            if isinstance(v, torch.Tensor):
                state[k] = v.to(self.device)
    self.is_offloaded = False

```

1.2.9 默认配置参数

```
[21]: def default_setup(manual=None):
    init_parameters = {
        # RoPE/MS-UPE
        'b_rope_or_upe': 10000,          # float: RoPE 或 UPE 的基数
        'head_ratio_upe': 2,             # float: UPE 头比例

        # MultiHeadAttention
        'num_heads': 8,                  # int: 注意力头数 H
        'd_model': 256,                  # int: Transformer 的维度 D
        'max_seq_len': 100,              # int: 模型支持的最大序列长度 (位置编码
        # 相关)
        'pe_type': ['learned_ape'],      # list: 位置编码类型 ('ape',
        # 'learned_ape', 'rope', 'ms_upe', 'alibi')

        # TransformerBlock
        'norm_type': 'layernorm',         # str: transformer_block 中的归一化类
        # 型 ('layernorm' 或 'rmsnorm')
        'ffn_type': 'gelu',               # str: transformer_block 中前馈网络激
        # 活函数类型 ('gelu' 或 'swiglu')

        # ToyModel
        'num_blocks': 20,                 # int: Transformer 的层数 b
        'loop': True,                     # bool: 是否权重共享
        'residual_gate': (1,1),           # tuple or str: transformer_block 之
        # 间传递残差的门控参数初始值, 使 `x=a*x+b*x_0` ((a,b) 或 'random')
        'residual_gate_type': 'fixed',    # str: 残差门控类型 ('fixed',
        # 'learnable_scalar', 'learnable_vector')
        'residual_random': (1,0.1),       # tuple (mean, std): 当
        # residual_gate='random' 时满足高斯分布 N(mean, std)

        # RegressionHead
        'd_x': 20,                        # int: 数据输入的特征维度
        'd_y': 1,                         # int: 数据输出的特征维度
        'bias': False,                    # bool: Regression Head 是否使用偏置
```

```

        'init_scale': None, # float or None: Regression Head 的初
        始化缩放因子 (None 表示使用默认初始化)

    # PredictionLoss
        'loss_type': 'mse', # str: 损失函数类型 ('mse' 或 'l1')
        'layer_weight_decay': 0.8, # float: 层权重衰减因子, 控制不同层的
        损失权重 (默认 0.8 表示从最后一层到第一层递减的权重, 1.0 表示不加权)
        'seq_weight_decay': 0.8, # float: 序列权重衰减因子, 控制不同时间
        步的损失权重 (默认 0.8 表示从最后一次预测到第一次预测递减的权重, 1.0 表示不加权)

    # LoopedTransformerExperiment
        'lr': 1e-4, # float: 学习率
        'gate_lr_ratio': 100, # float: 当使用残差门控时, 门控参数的
        学习率相对于模型其他参数的倍数
        'seed': 42, # int or None: 手动随机种子, 确保实验
        可复现
        'experiment_name': 'Experiment', # str or None: 实验名称 (None 表示不打
        印设备和实验初始化信息)
        'timing': True, # bool: 是否打印初始化时间
        'optimizer_type': 'adamw', # str: 优化器类型 ('adam' 或 'sgd'或
        ↪ 'adamw')
        'wd_adamw': 0.01, # float: AdamW 优化器的权重衰减系数,
        仅当 optimizer_type='adamw'时有效
        'task': 'regression', # str: 任务类型 ('regression')
    }

    train_parameters = {
    # MultiHeadAttention
        'batch_size': 64, # int: 数据批次大小
        'seq_len': 80, # int: 数据序列长度

    # ToyModel
        'num_eff': 15, # int: 有效层数, 即参与误差计算、参与梯
        度反向传播的层数  $T=b-b_0$ 

```

```

# dataloader
'sink_padding': None, # int or None: seq 最前面填充的 sink
↪token 组数 (None 表示不使用 sink padding)
'd_hidden': 64, # int: 非线性数据生成器中隐藏层的维度,
仅当 data_type='nonlinear'时有效
'function_callable': lambda x: F.relu(x), # function: 非线性数据生成器中
使用的非线性函数, 仅当 data_type='nonlinear'时有效

# LoopedTransformerExperiment
'epochs': 100, # int: 训练轮数
'data_type': 'linear', # str: 回归数据类型 ('linear')
'scheduler_type': None, # str or None: 学习率调节器类型
↪('cosine' 或 'step' 或 None)
'lr_scale': 0.1, # float: 学习率调节器的缩放因子, 仅当
scheduler_type 不为 None 时有效
'step_size_scheduler': 10, # int: StepLR 调度器的步长, 仅当
scheduler_type='step'时有效
'scheduled_training': False, # bool: 是否使用计划训练, 即随着训练的
进行逐渐增加参与的层数  $b=epoch+num\_eff$  , 直到达到最大层数  $num\_blocks$ 
'print_every': 20, # int or None: 打印频率/epoch (None
表示不打印训练过程中的损失信息)
'timing': True, # bool: 是否打印训练时间
}
if manual is not None:
    for key,value in manual.items():
        if key in init_parameters:
            init_parameters[key]=value
        elif key in train_parameters:
            train_parameters[key]=value
        else:
            print(f"Warning:key {key} do not exist in init_parameters and
↪train_parameters.")
    return init_parameters, train_parameters

```

1.2.10 显存检测

```
[22]: def print_vram_usage(tag="", peak=None):
    if torch.backends.mps.is_available():
        # MPS 专用 API (Apple Silicon)
        allocated = torch.mps.current_allocated_memory() / (1024 ** 3)
        driver_alloc = torch.mps.driver_allocated_memory() / (1024 ** 3)
        if peak is not None:
            print(f"[{tag}] 显存占用 -> PyTorch 分配: {peak[0]:.2f} GB | Metal  
驱动分配: {peak[1]:.2f} GB")
        else:
            print(f"[{tag}] 显存占用 -> PyTorch 分配: {allocated:.2f} GB | Metal  
驱动分配: {driver_alloc:.2f} GB")

    elif torch.cuda.is_available():
        # CUDA 专用 API
        allocated = torch.cuda.memory_allocated() / (1024 ** 3)
        peak = torch.cuda.max_memory_allocated() / (1024 ** 3)
        reserved = torch.cuda.memory_reserved() / (1024 ** 3)
        total = torch.cuda.get_device_properties(0).total_memory / (1024 ** 3)
        free = total - reserved
        print(f"[{tag}] 显存占用 -> 已分配: {allocated:.2f} GB | 峰值: {peak:.  
↪2f} GB | 缓存池: {reserved:.2f} GB | 剩余可用: {free:.2f} GB")

    else:
        print(f"[{tag}] 当前运行在 CPU, 占用主板内存。")
```

1.3 实验台搭建

```
[ ]: class ExperimentTable:
    '''
    Attributes:
        num_experiments: int, 实验数量
        init_parameters: list of dict, 在 __init__() 方法中传入, 用于指定每个实验  
的初始化参数, 可以覆盖默认参数
        train_parameters: list of dict, 在 __init__() 方法中传入, 用于指定每个实  
验的训练参数, 可以覆盖默认参数
```

result_lists: list of tuple of list, 在 `run()` 方法中传入, 用于指定需要展示的结果名称和对应的横坐标类型。

results_groups: list of tuple of list, 包含不同模式 (如 'train' 和 'evaluate') 下的结果列表, 每个结果列表是一个包含所有实验结果的列表, 如 `[('train', [experiment1_train_results, experiment2_train_results, ...]), ('evaluate', [experiment1_evaluate_results, experiment2_evaluate_results, ...])]`

'''

```
def __init__(self, params_groups:list[dict], manual=None):
```

'''

Args:

params_groups: list of dict, 每个 `dict` 包含一个实验的参数设置, 可以覆盖默认参数,

如 `[{residual_gate: (0.5, 0.5), experiment_name: 'Experiment 1'}, {residual_gate: (0.8, 0.2), experiment_name: 'Experiment 2'}]`

manual: dict, 全局默认参数修改

Attributes:

num_experiments: int, 实验数量

init_parameters: list of dict, 每个 `dict` 包含一个实验的初始化参数设置

train_parameters: list of dict, 每个 `dict` 包含一个实验的训练参数设置

'''

```
init, train = default_setup(manual=manual)
```

```
self.num_experiments = len(params_groups)
```

```
self.init_parameters = [copy.deepcopy(init) for _ in range(self.  
num_experiments)]
```

```
self.train_parameters = [copy.deepcopy(train) for _ in range(self.  
num_experiments)]
```

```
# 自动分类传入的参数以更新默认参数
```

```
for i, params in enumerate(params_groups):
```

```
    for key, value in params.items():
```

```
        if key in self.init_parameters[i]:
```

```
            self.init_parameters[i][key] = value
```

```
        elif key in self.train_parameters[i]:
```

```

        self.train_parameters[i][key] = value
    else:
        raise ValueError(f"Unknown parameter: {key}")

    def run(self, result_lists: list[tuple[list[str], str, int]],
    ↪ modes=['train'], parallel_workers=1):
        """
        Args:
            result_lists: list of tuple of list, 每个 tuple 包含一组实验测量的数据、对应的横坐标类型和 baseline 索引（不写表示不使用 baseline），每个 list 中的字符串表示需要展示的结果名称，必须是 LoopedTransformerExperiment.get_results() 中返回的结果名称，
                如 [(['loss_history', 'residual_gate_history_a_mean',
                ↪ 'residual_gate_history_b_mean'], 'epoch', 0), (['final_loss',
                ↪ 'final_residual_gate_a_mean', 'final_residual_gate_b_mean'], 'experiment')]

            modes: list of str, 表示需要运行的模式，可以是 'train' 和 'evaluate'
            ↪ 的任意组合，'train' 表示需要训练模型并记录训练结果，'evaluate' 表示需要在训练完成后进行评估并记录评估结果

            parallel_workers: int, 表示并行运行实验的工作线程数。

        Attributes:
            result_lists: 同 Args
            results_groups: dict, 包含不同模式（如'train'和'evaluate'）下的结果列表，每个 value 是一个 list of dict, 每个 dict 是一个实验的结果，为 metric 和 value 的键值对。

            experiments: list of LoopedTransformerExperiment 对象，已转移到 CPU 以节省 GPU 显存
        """
        # 创建实验对象并运行
        result_set = set().union(*[item[0] for item in result_lists]) # 兼容长度为 2 或 3 的 tuple, 获取所有需要展示的结果名称的集合
        self.result_lists = result_lists
        self.results_groups = {'train': [None]*self.num_experiments, 'evaluate':
        ↪ [None]*self.num_experiments}

```

```

self.experiments=[None]*self.num_experiments
print_lock = threading.Lock() # 用于在线程中安全地打印日志
vram_printed = False
for i in range(self.num_experiments):
    experiment = LoopedTransformerExperiment(**self.init_parameters[i])
    experiment.offload_to_cpu() # 初始化后将模型移回 CPU, 节省 GPU 显存
    self.experiments[i] = experiment
def single_train(i, parallel=False):
    nonlocal vram_printed
    experiment = self.experiments[i]
    experiment.load_to_device() # 训练前将模型加载到 GPU
    if parallel:
        self.train_parameters[i]['timing']=False
        self.train_parameters[i]['print_every']=None
    if parallel:
        with print_lock:
            if not vram_printed:
                print_vram_usage(tag=f"Parallel 峰值显存检测")
                vram_printed = True
    else:
        print_vram_usage(tag=f"Experiment {i+1} 训练前")
    for mode in modes:
        if mode == 'train':
            experiment.train(**self.train_parameters[i])
            self.results_groups['train'][i] = experiment.
↪get_results(result_set)
            if parallel:
                with print_lock:
                    print_vram_usage(tag=f"Experiment {i+1} 训练峰值",
↪peak=(experiment.max_torch_vram, experiment.max_metal_vram))
            else:
                print_vram_usage(tag=f"Experiment {i+1} 训练峰值",
↪peak=(experiment.max_torch_vram, experiment.max_metal_vram))
            elif mode == 'evaluate':

```



```

        eval_parameters = {k:v for k,v in self.train_parameters[i].
↪items() if k in ['batch_size', 'seq_len', 'data_type', 'sink_padding',
↪'d_hidden', 'function_callable']}

        if 'loss_type' in self.init_parameters[i]:
            eval_parameters['loss_type'] = self.
↪init_parameters[i]['loss_type'] # 确保评估使用与训练相同类型的损失函数

            self.results_groups['evaluate'][i] = experiment.
↪evaluate(**eval_parameters)

            experiment.offload_to_cpu() # 训练完成后将模型和结果移回 CPU, 节省 GPU
显存

        self.experiments[i] = experiment
        if torch.cuda.is_available():
            torch.cuda.empty_cache()
        if torch.backends.mps.is_available():
            torch.mps.empty_cache()
        if not parallel:
            print_vram_usage(tag=f"Experiment {i+1} 清理后")
            print(f"Experiment {i+1}/{self.num_experiments} completed.\n")

        print_vram_usage(tag=f"本底显存检测")
        if parallel_workers == 1:
            for i in range(self.num_experiments):
                single_train(i, parallel=False)
            elif isinstance(parallel_workers, int) and parallel_workers > 1:
                print(f"Running {self.num_experiments} experiments in parallel with
↪{parallel_workers} workers...")
                with concurrent.futures.
↪ThreadPoolExecutor(max_workers=parallel_workers) as executor:
                    executor.map(lambda i: single_train(i, parallel=True),
↪range(self.num_experiments))
            else:
                raise ValueError("parallel_workers must be a positive integer.")

        def plot(self, compare_experiments=True,
            subplot_shape=(1,-1),figure_size=None,suptitle='Looped Transformer
↪Experiment Results',

```

```

        colors=['blue', 'red', 'green', 'yellow', 'cyan', 'orange', 'purple', 'brown']):
'''
    Args:
        compare_experiments: bool, True 表示在一张图中比较不同实验的结果，每张图仅比较一个指标；False 表示每个实验单独成图，每张图仅比较一个实验的多个指标（第一个指标使用左轴，第二个及以后的指标使用共享 x 轴的右轴）

        subplot_shape: tuple of int, 表示子图的行数和列数，(-1 表示自动计算)，如 (1, -1) 表示所有图在一行，(-1, 2) 表示每行两个图，自动计算行数

        figure_size: tuple of float, 画布大小，单位为英寸，如 (15, 10)。默认值 None 表示使用 (8,6)*subplot_shape 的大小

        suptitle: str, 整体图的标题

        colors: list of str, 画图使用的颜色列表，每张图从头开始循环使用这些颜色
'''
# 画图
if compare_experiments:
    num_plots = len(self.result_lists) # 需要画的图的总数量
else:
    num_plots = len(self.result_lists)*self.num_experiments # 需要画的图的总数量

if subplot_shape == (-1, -1):
    raise ValueError("Invalid subplot_shape: both dimensions cannot be -1. Please specify at least one dimension or provide a valid shape.")
if subplot_shape[1] == -1:
    subplot_shape = (subplot_shape[0], math.ceil(num_plots / subplot_shape[0])) # 自动计算列数
elif subplot_shape[0] == -1:
    subplot_shape = (math.ceil(num_plots / subplot_shape[1]), subplot_shape[1]) # 自动计算行数
if subplot_shape[0] * subplot_shape[1] < num_plots:
    raise ValueError(f"subplot_shape {subplot_shape} is too small for the number of plots {num_plots}. Please increase the dimensions.")

```

```

if self.results_groups['train'][0] is not None:
    results = self.results_groups['train']
    if figure_size is None:
        figure_size = (8*subplot_shape[1], 6*subplot_shape[0])
    fig,axs = plt.subplots(*subplot_shape, figsize=figure_size)
    axs_flat = np.atleast_1d(axs).flatten() # 将 axs 转换为一维数组，方便索引

    index=0
    for item in self.result_lists:
        result_list = item[0] # 需要展示的结果名称列表
        x = item[1] # 横坐标类型 ('epoch' 或 'experiment')
        baseline_index = item[2] if len(item) > 2 else None # 基线索引 (可选)，表示用于计算相对值的基线实验索引
        baseline_name=None
        if x=='epoch': # 表示画横坐标是 epoch 的曲线
            if compare_experiments:
                for metric in result_list:
                    if baseline_index is not None and 0 <= baseline_index < self.num_experiments:
                        baseline_name = self.init_parameters[baseline_index].get('experiment_name', f'Experiment_{baseline_index+1}')
                        baseline_data = np.array(results[baseline_index][metric])
                        for i in range(self.num_experiments):
                            exp_name = self.init_parameters[i].get('experiment_name', f'Experiment {i+1}')
                            y_data = np.array(results[i][metric])
                            if baseline_name is not None:
                                y_data = y_data - baseline_data # 计算相对于基线的差值
                            if i == baseline_index:
                                exp_name += ' (Baseline)'
                            axs_flat[index].plot(y_data, label=exp_name, color=colors[i%len(colors)])

```

```

        axs_flat[index].set_xlabel('Epoch')
        if baseline_name is not None:
            axs_flat[index].set_ylabel(f"{metric} (relative_
↳to {baseline_name})")

        else:
            axs_flat[index].set_ylabel(metric)
            axs_flat[index].tick_params(axis='y')
            axs_flat[index].set_title('Comparison of ' + metric)
            axs_flat[index].legend(loc='upper right')
            axs_flat[index].grid(True)
            index += 1

    else:
        if baseline_index is not None and 0 <= baseline_index <
↳self.num_experiments:

            baseline_name = self.
↳init_parameters[baseline_index].get('experiment_name', f'Experiment_
↳{baseline_index+1}')

            baseline_data = np.
↳array(results[baseline_index][result_list[0]])

            for i in range(self.num_experiments):
                color_index = 0
                y_data = np.array(results[i][result_list[0]])
                if baseline_name is not None:
                    y_data = y_data - baseline_data # 计算相对于基
线的差值

                    if i == baseline_index:
                        exp_name += ' (Baseline)'
                    axs_flat[index].plot(y_data, label=result_list[0],
↳color=colors[color_index%len(colors)])
                    axs_flat[index].set_xlabel('Epoch')
                    if baseline_name is not None:
                        axs_flat[index].set_ylabel(f"{result_list[0]}_
↳(relative to {baseline_name})", color=colors[color_index%len(colors)])
                    else:
                        axs_flat[index].set_ylabel(result_list[0],
↳color=colors[color_index%len(colors)])

```

```

        axs_flat[index].tick_params(axis='y',
↪labelcolor=colors[color_index%len(colors)])
        color_index += 1
        if len(result_list) > 1:
            ax_twin = axs_flat[index].twinx() # 创建共享 x
轴但独立 y 轴的第二个坐标轴

            primary_color = colors[color_index%len(colors)]
            for j in range(1, len(result_list)):
                y_data = np.
↪array(results[i][result_list[j]])

                if baseline_name is not None:
                    y_data = y_data - baseline_data # 计算
相对于基线的差值

                    if i == baseline_index:
                        exp_name += ' (Baseline)'
                    ax_twin.plot(y_data, label=result_list[j],
↪color=colors[color_index%len(colors)])
                    color_index += 1
                    ax_twin.set_ylabel(' and '.join(result_list[1:
↪]), color=primary_color)

                    ax_twin.tick_params(axis='y',
↪labelcolor=primary_color)

                    ax_twin.legend(loc='upper right')
                    axs_flat[index].set_title(self.
↪init_parameters[i]['experiment_name']+'-'+ ' and '.join(result_list))
                    axs_flat[index].legend(loc='upper left')
                    axs_flat[index].grid(True)
                    index += 1

            elif x=='experiment': # 表示画不同实验对比的柱状图
                pass
            # 如果绘制的图的数量少于子图的总数量, 则隐藏多余的子图
            for i in range(index, len(axs_flat)):
                axs_flat[i].set_visible(False)
            if self.results_groups['evaluate'][0] is not None:
                pass
            if supitle is not None:

```

```
plt.suptitle(suptitle)
plt.tight_layout()
plt.show()
```

2 Linear Regression Experiments

2.1 实验数据观测

2.1.1 绘制 Loss 下降曲线

```
[23]: loss_table = ExperimentTable(params_groups=[{'experiment_name': '👉',
↪ 'loss_history'}])
loss_table.run(result_lists=[(['loss_history'], 'epoch')])
loss_table.plot(figsize=(8,6))
```

loss_history initialized in 0.02 seconds. Using device: mps

[本底显存检测] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.02 GB

[Experiment 1 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.02 GB

Epoch 5/20, Loss: 63.3171, current_blocks: 19

Epoch 10/20, Loss: 35.1716, current_blocks: 20

Epoch 15/20, Loss: 31.4720, current_blocks: 20

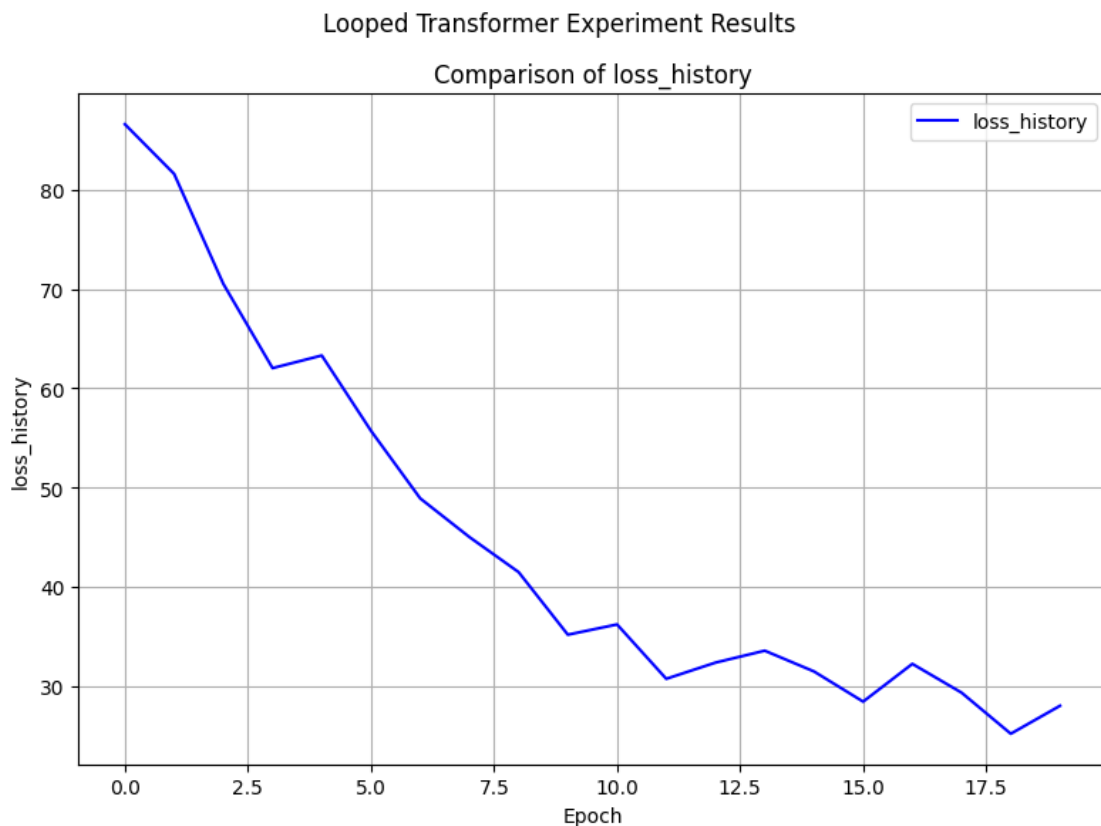
Epoch 20/20, Loss: 28.0077, current_blocks: 20

Training completed in 6.75 seconds.

[Experiment 1 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.15 GB

[Experiment 1 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB

Experiment 1/1 completed.



2.1.2 绘制 y_{pred} 和 y_{true} 的变化曲线

```
[75]: norm_table = ExperimentTable(params_groups=[{'epochs': 1000, 'experiment_name': 'Looped Transformer', 'y_norm_error_history', 'print_every': 200}])
norm_table.run(result_lists=[(['y_norm_error_history'], 'epoch')])
norm_table.plot.figure_size=(8,6),compare_experiments=False)
```

y_norm_error_history initialized in 0.02 seconds. Using device: mps

[本底显存检测] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB

[Experiment 1 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB

Epoch 200/1000, Loss: 18.8248, current_blocks: 20

Epoch 400/1000, Loss: 19.7988, current_blocks: 20

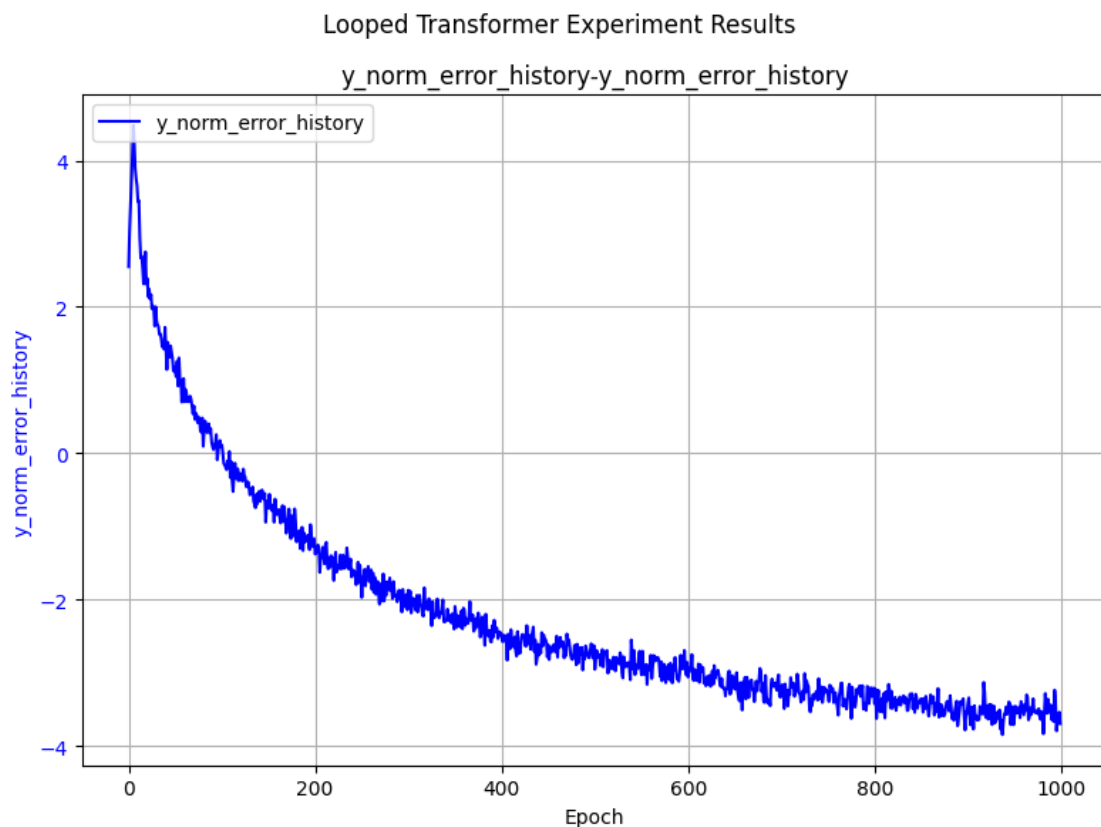
Epoch 600/1000, Loss: 17.7491, current_blocks: 20

Epoch 800/1000, Loss: 20.9858, current_blocks: 20

Epoch 1000/1000, Loss: 20.9834, current_blocks: 20

Training completed in 451.35 seconds.

[Experiment 1 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.16 GB
[Experiment 1 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB
Experiment 1/1 completed.



2.2 对比实验

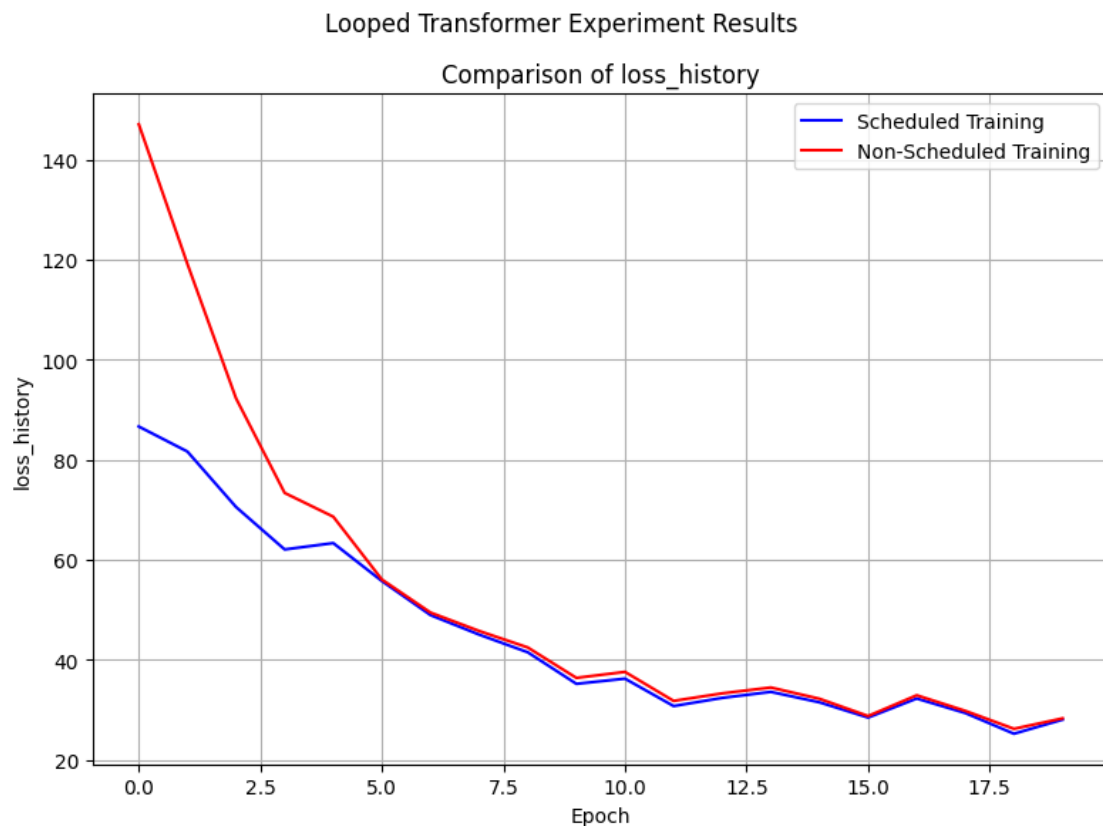
2.2.1 Scheduled Training vs Non-Scheduled Training 对比实验

```
[76]: table = ExperimentTable(params_groups=[{'scheduled_training': True,
      ↪ 'experiment_name': 'Scheduled Training'},
      {'scheduled_training': False,
      ↪ 'experiment_name': 'Non-Scheduled Training'}])
table.run(result_lists=[(['loss_history'], 'epoch')])
table.plot(figure_size=(8, 6), compare_experiments=True)
```

Scheduled Training initialized in 0.02 seconds. Using device: mps

Non-Scheduled Training initialized in 0.01 seconds. Using device: mps
[本底显存检测] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.05 GB
[Experiment 1 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.05 GB
Epoch 5/20, Loss: 63.3171, current_blocks: 19
Epoch 10/20, Loss: 35.1716, current_blocks: 20
Epoch 15/20, Loss: 31.4720, current_blocks: 20
Epoch 20/20, Loss: 28.0077, current_blocks: 20
Training completed in 9.75 seconds.
[Experiment 1 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.16 GB
[Experiment 1 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB
Experiment 1/2 completed.

[Experiment 2 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.05 GB
Epoch 5/20, Loss: 68.5955, current_blocks: 20
Epoch 10/20, Loss: 36.3633, current_blocks: 20
Epoch 15/20, Loss: 32.1849, current_blocks: 20
Epoch 20/20, Loss: 28.2660, current_blocks: 20
Training completed in 9.81 seconds.
[Experiment 2 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.13 GB
[Experiment 2 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB
Experiment 2/2 completed.



2.2.2 不同 Positional Embedding 对比实验

```
[79]: pe_table = ExperimentTable(params_groups=[
    {'pe_type': ['learned_ape'], 'experiment_name': 'Learned APE'},
    {'pe_type': ['ape'], 'experiment_name': 'APE'},
    {'pe_type': ['alibi'], 'experiment_name': 'ALiBi'},
    {'pe_type': ['rope'], 'experiment_name': 'RoPE'},
    {'pe_type': ['ms_upe'], 'experiment_name': 'MS-UPE'},
    {'pe_type': ['ms_upe', 'learned_ape'], 'experiment_name': 'MS-UPE + Learned_
    ↪APE'},
], manual={'epochs': 100, 'print_every': 20})
pe_table.run(result_lists=[
    (['loss_history'], 'epoch'),
    (['loss_history'], 'epoch', 2) # 以 ALiBi 为基线
])
```

```
pe_table.plot(figure_size=(12, 6))
```

Learned APE initialized in 0.02 seconds. Using device: mps

APE initialized in 0.01 seconds. Using device: mps

ALiBi initialized in 0.01 seconds. Using device: mps

RoPE initialized in 0.01 seconds. Using device: mps

MS-UPE initialized in 0.01 seconds. Using device: mps

MS-UPE + Learned APE initialized in 0.01 seconds. Using device: mps

[本底显存检测] 显存占用 -> PyTorch 分配: 3.10 GB | Metal 驱动分配: 4.19 GB

[Experiment 1 训练前] 显存占用 -> PyTorch 分配: 3.11 GB | Metal 驱动分配: 4.19 GB

Epoch 20/100, Loss: 28.0077, current_blocks: 20

Epoch 40/100, Loss: 20.8999, current_blocks: 20

Epoch 60/100, Loss: 20.2006, current_blocks: 20

Epoch 80/100, Loss: 18.3025, current_blocks: 20

Epoch 100/100, Loss: 21.6682, current_blocks: 20

Training completed in 47.76 seconds.

[Experiment 1 训练峰值] 显存占用 -> PyTorch 分配: 3.13 GB | Metal 驱动分配: 5.22 GB

[Experiment 1 清理后] 显存占用 -> PyTorch 分配: 3.10 GB | Metal 驱动分配: 4.21 GB

Experiment 1/6 completed.

[Experiment 2 训练前] 显存占用 -> PyTorch 分配: 3.11 GB | Metal 驱动分配: 4.21 GB

Epoch 20/100, Loss: 25.4152, current_blocks: 20

Epoch 40/100, Loss: 20.2102, current_blocks: 20

Epoch 60/100, Loss: 19.7146, current_blocks: 20

Epoch 80/100, Loss: 17.6053, current_blocks: 20

Epoch 100/100, Loss: 21.3824, current_blocks: 20

Training completed in 48.05 seconds.

[Experiment 2 训练峰值] 显存占用 -> PyTorch 分配: 3.13 GB | Metal 驱动分配: 5.21 GB

[Experiment 2 清理后] 显存占用 -> PyTorch 分配: 3.10 GB | Metal 驱动分配: 4.21 GB

Experiment 2/6 completed.

[Experiment 3 训练前] 显存占用 -> PyTorch 分配: 3.11 GB | Metal 驱动分配: 4.21 GB

Epoch 20/100, Loss: 24.0522, current_blocks: 20

Epoch 40/100, Loss: 20.8352, current_blocks: 20

Epoch 60/100, Loss: 19.5621, current_blocks: 20

Epoch 80/100, Loss: 17.1478, current_blocks: 20

Epoch 100/100, Loss: 21.7769, current_blocks: 20

Training completed in 55.24 seconds.

[Experiment 3 训练峰值] 显存占用 -> PyTorch 分配: 3.13 GB | Metal 驱动分配: 5.23 GB

[Experiment 3 清理后] 显存占用 -> PyTorch 分配: 3.10 GB | Metal 驱动分配: 4.21 GB

Experiment 3/6 completed.

[Experiment 4 训练前] 显存占用 -> PyTorch 分配: 3.11 GB | Metal 驱动分配: 4.21 GB

Epoch 20/100, Loss: 23.8768, current_blocks: 20

Epoch 40/100, Loss: 20.4678, current_blocks: 20

Epoch 60/100, Loss: 19.6902, current_blocks: 20

Epoch 80/100, Loss: 17.1698, current_blocks: 20

Epoch 100/100, Loss: 21.6961, current_blocks: 20

Training completed in 68.12 seconds.

[Experiment 4 训练峰值] 显存占用 -> PyTorch 分配: 3.13 GB | Metal 驱动分配: 5.22 GB

[Experiment 4 清理后] 显存占用 -> PyTorch 分配: 3.10 GB | Metal 驱动分配: 4.21 GB

Experiment 4/6 completed.

[Experiment 5 训练前] 显存占用 -> PyTorch 分配: 3.11 GB | Metal 驱动分配: 4.21 GB

Epoch 20/100, Loss: 23.8994, current_blocks: 20

Epoch 40/100, Loss: 20.4356, current_blocks: 20

Epoch 60/100, Loss: 19.6087, current_blocks: 20

Epoch 80/100, Loss: 17.1837, current_blocks: 20

Epoch 100/100, Loss: 21.6859, current_blocks: 20

Training completed in 52.83 seconds.

[Experiment 5 训练峰值] 显存占用 -> PyTorch 分配: 3.13 GB | Metal 驱动分配: 5.22 GB

[Experiment 5 清理后] 显存占用 -> PyTorch 分配: 3.10 GB | Metal 驱动分配: 4.21 GB

Experiment 5/6 completed.

[Experiment 6 训练前] 显存占用 -> PyTorch 分配: 3.11 GB | Metal 驱动分配: 4.21 GB

Epoch 20/100, Loss: 28.1398, current_blocks: 20

Epoch 40/100, Loss: 20.6823, current_blocks: 20

Epoch 60/100, Loss: 20.1465, current_blocks: 20

Epoch 80/100, Loss: 18.4156, current_blocks: 20

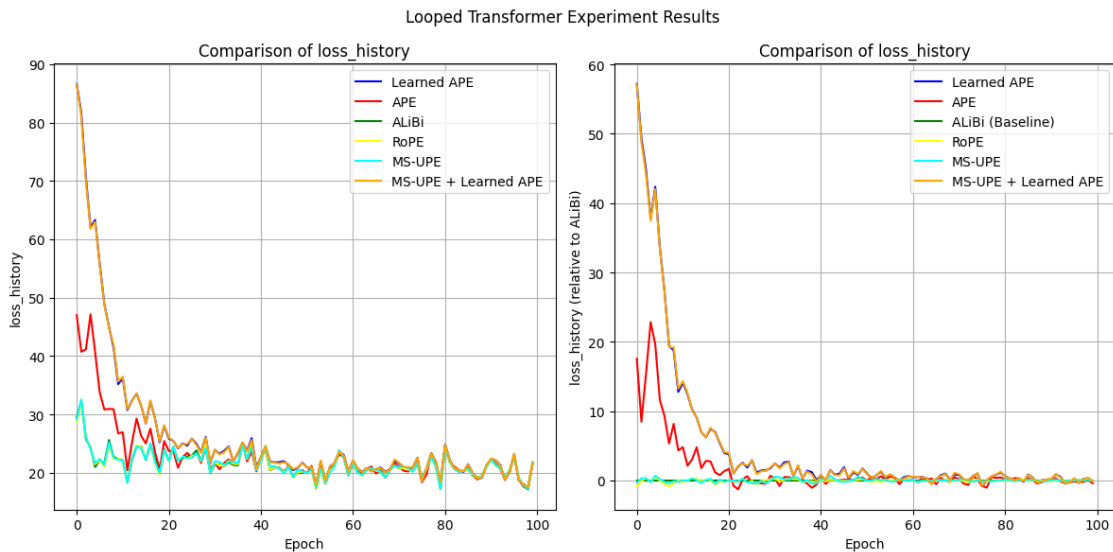
Epoch 100/100, Loss: 21.6613, current_blocks: 20

Training completed in 54.00 seconds.

[Experiment 6 训练峰值] 显存占用 -> PyTorch 分配: 3.13 GB | Metal 驱动分配: 5.23 GB

[Experiment 6 清理后] 显存占用 -> PyTorch 分配: 3.10 GB | Metal 驱动分配: 4.21 GB

Experiment 6/6 completed.



2.2.3 sink padding 的有无对比实验

```
[80]: sink_table = ExperimentTable(params_groups=[
    {'experiment_name': 'Without Sink Padding'},
    {'experiment_name': 'With Sink Padding (1)', 'sink_padding': 1},
    {'experiment_name': 'With Sink Padding (2)', 'sink_padding': 2},
    {'experiment_name': 'With Sink Padding (5)', 'sink_padding': 5},
    {'experiment_name': 'With Sink Padding (10)', 'sink_padding': 10},
], manual={'epochs': 100, 'print_every': 20})
sink_table.run(result_lists=[
    (['loss_history'], 'epoch'),
    (['loss_history'], 'epoch', 0), # 以 Without Sink Padding 为基线
])
sink_table.plot(figure_size=(12, 6))
```

Without Sink Padding initialized in 0.02 seconds. Using device: mps
 With Sink Padding (1) initialized in 0.01 seconds. Using device: mps
 With Sink Padding (2) initialized in 0.01 seconds. Using device: mps
 With Sink Padding (5) initialized in 0.01 seconds. Using device: mps
 With Sink Padding (10) initialized in 0.01 seconds. Using device: mps

[本底显存检测] 显存占用 -> PyTorch 分配: 3.10 GB | Metal 驱动分配: 4.21 GB
[Experiment 1 训练前] 显存占用 -> PyTorch 分配: 3.11 GB | Metal 驱动分配: 4.21 GB
Epoch 20/100, Loss: 28.0077, current_blocks: 20
Epoch 40/100, Loss: 20.8999, current_blocks: 20
Epoch 60/100, Loss: 20.2006, current_blocks: 20
Epoch 80/100, Loss: 18.3025, current_blocks: 20
Epoch 100/100, Loss: 21.6682, current_blocks: 20
Training completed in 50.96 seconds.
[Experiment 1 训练峰值] 显存占用 -> PyTorch 分配: 3.13 GB | Metal 驱动分配: 5.22 GB
[Experiment 1 清理后] 显存占用 -> PyTorch 分配: 3.10 GB | Metal 驱动分配: 4.21 GB
Experiment 1/5 completed.

[Experiment 2 训练前] 显存占用 -> PyTorch 分配: 3.11 GB | Metal 驱动分配: 4.21 GB
Epoch 20/100, Loss: 28.1106, current_blocks: 20
Epoch 40/100, Loss: 20.9680, current_blocks: 20
Epoch 60/100, Loss: 20.6434, current_blocks: 20
Epoch 80/100, Loss: 17.8341, current_blocks: 20
Epoch 100/100, Loss: 21.5738, current_blocks: 20
Training completed in 53.92 seconds.
[Experiment 2 训练峰值] 显存占用 -> PyTorch 分配: 3.13 GB | Metal 驱动分配: 5.22 GB
[Experiment 2 清理后] 显存占用 -> PyTorch 分配: 3.10 GB | Metal 驱动分配: 4.21 GB
Experiment 2/5 completed.

[Experiment 3 训练前] 显存占用 -> PyTorch 分配: 3.11 GB | Metal 驱动分配: 4.21 GB
Epoch 20/100, Loss: 29.5266, current_blocks: 20
Epoch 40/100, Loss: 20.6409, current_blocks: 20
Epoch 60/100, Loss: 20.1771, current_blocks: 20
Epoch 80/100, Loss: 18.3634, current_blocks: 20
Epoch 100/100, Loss: 21.4438, current_blocks: 20
Training completed in 53.87 seconds.
[Experiment 3 训练峰值] 显存占用 -> PyTorch 分配: 3.13 GB | Metal 驱动分配: 5.22 GB
[Experiment 3 清理后] 显存占用 -> PyTorch 分配: 3.10 GB | Metal 驱动分配: 4.21 GB
Experiment 3/5 completed.

[Experiment 4 训练前] 显存占用 -> PyTorch 分配: 3.11 GB | Metal 驱动分配: 4.21 GB
Epoch 20/100, Loss: 26.8264, current_blocks: 20
Epoch 40/100, Loss: 20.8156, current_blocks: 20

Epoch 60/100, Loss: 20.2012, current_blocks: 20

Epoch 80/100, Loss: 18.7726, current_blocks: 20

Epoch 100/100, Loss: 21.6547, current_blocks: 20

Training completed in 57.91 seconds.

[Experiment 4 训练峰值] 显存占用 -> PyTorch 分配: 3.13 GB | Metal 驱动分配: 6.22 GB

[Experiment 4 清理后] 显存占用 -> PyTorch 分配: 3.11 GB | Metal 驱动分配: 4.21 GB

Experiment 4/5 completed.

[Experiment 5 训练前] 显存占用 -> PyTorch 分配: 3.11 GB | Metal 驱动分配: 4.21 GB

Epoch 20/100, Loss: 27.0638, current_blocks: 20

Epoch 40/100, Loss: 21.4043, current_blocks: 20

Epoch 60/100, Loss: 20.5190, current_blocks: 20

Epoch 80/100, Loss: 18.5852, current_blocks: 20

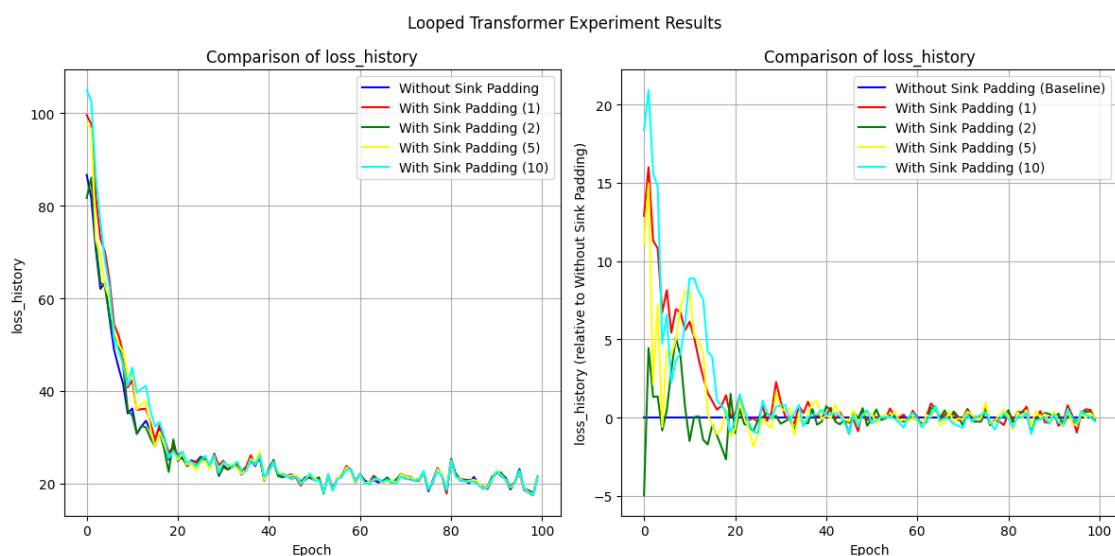
Epoch 100/100, Loss: 21.4375, current_blocks: 20

Training completed in 66.44 seconds.

[Experiment 5 训练峰值] 显存占用 -> PyTorch 分配: 3.14 GB | Metal 驱动分配: 6.26 GB

[Experiment 5 清理后] 显存占用 -> PyTorch 分配: 3.11 GB | Metal 驱动分配: 4.21 GB

Experiment 5/5 completed.



2.2.4 Residual Gate 对比实验

```
[27]: gate_table = ExperimentTable(params_groups=[
    {'experiment_name': 'No Residual Gate', 'residual_gate': (0, 0),
    ↪ 'residual_gate_type': 'fixed'},
    {'experiment_name': 'Residual Gate (1, 0.5)', 'residual_gate': (1, 0.5),
    ↪ 'residual_gate_type': 'learnable_scalar'},
    {'experiment_name': 'Residual Gate (1, 0.5) Vector', 'residual_gate': (1, 0.
    ↪ 5), 'residual_gate_type': 'learnable_vector'},
    {'experiment_name': 'Residual Gate (1, 0)', 'residual_gate': (1, 0),
    ↪ 'residual_gate_type': 'learnable_scalar'},
    {'experiment_name': 'Residual Gate (1, 0) Vector', 'residual_gate': (1, 0),
    ↪ 'residual_gate_type': 'learnable_vector'},
    {'experiment_name': 'Random Learnable Scalar', 'residual_gate': 'random',
    ↪ 'residual_gate_type': 'learnable_scalar', 'residual_random': (0.9, 0.1)},
    {'experiment_name': 'Random Learnable Vector', 'residual_gate': 'random',
    ↪ 'residual_gate_type': 'learnable_vector', 'residual_random': (0.9, 0.1)},
], manual={'gate_lr_ratio': 100, 'scheduler_type': 'cosine', 'epochs': 50,
    ↪ 'print_every': 25})
gate_table.run(result_lists=[
    (['loss_history'], 'epoch')
])
gate_table.plot(figure_size=(8, 6))
```

No Residual Gate initialized in 0.01 seconds. Using device: mps

Residual Gate (1, 0.5) initialized in 0.01 seconds. Using device: mps

Residual Gate (1, 0.5) Vector initialized in 0.01 seconds. Using device: mps

Residual Gate (1, 0) initialized in 0.01 seconds. Using device: mps

Residual Gate (1, 0) Vector initialized in 0.01 seconds. Using device: mps

Random Learnable Scalar initialized in 0.01 seconds. Using device: mps

Random Learnable Vector initialized in 0.01 seconds. Using device: mps

[本底显存检测] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB

[Experiment 1 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB

Epoch 25/50, Loss: 20.7067, current_blocks: 20

Epoch 50/50, Loss: 19.7848, current_blocks: 20

Training completed in 19.79 seconds.

[Experiment 1 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.16 GB

[Experiment 1 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB
Experiment 1/7 completed.

[Experiment 2 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB
Epoch 25/50, Loss: 21.0445, current_blocks: 20
Epoch 50/50, Loss: 19.9295, current_blocks: 20
Training completed in 20.28 seconds.

[Experiment 2 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.15 GB
[Experiment 2 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB
Experiment 2/7 completed.

[Experiment 3 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB
Epoch 25/50, Loss: 21.1970, current_blocks: 20
Epoch 50/50, Loss: 20.4095, current_blocks: 20
Training completed in 20.98 seconds.

[Experiment 3 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.15 GB
[Experiment 3 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB
Experiment 3/7 completed.

[Experiment 4 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB
Epoch 25/50, Loss: 21.1870, current_blocks: 20
Epoch 50/50, Loss: 19.9217, current_blocks: 20
Training completed in 20.64 seconds.

[Experiment 4 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.15 GB
[Experiment 4 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB
Experiment 4/7 completed.

[Experiment 5 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB
Epoch 25/50, Loss: 20.5020, current_blocks: 20
Epoch 50/50, Loss: 19.9380, current_blocks: 20
Training completed in 21.49 seconds.

[Experiment 5 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.16 GB
[Experiment 5 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB
Experiment 5/7 completed.

[Experiment 6 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB
Epoch 25/50, Loss: 20.6064, current_blocks: 20

Epoch 50/50, Loss: 19.8823, current_blocks: 20

Training completed in 21.05 seconds.

[Experiment 6 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.16 GB

[Experiment 6 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB

Experiment 6/7 completed.

[Experiment 7 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB

Epoch 25/50, Loss: 22.2538, current_blocks: 20

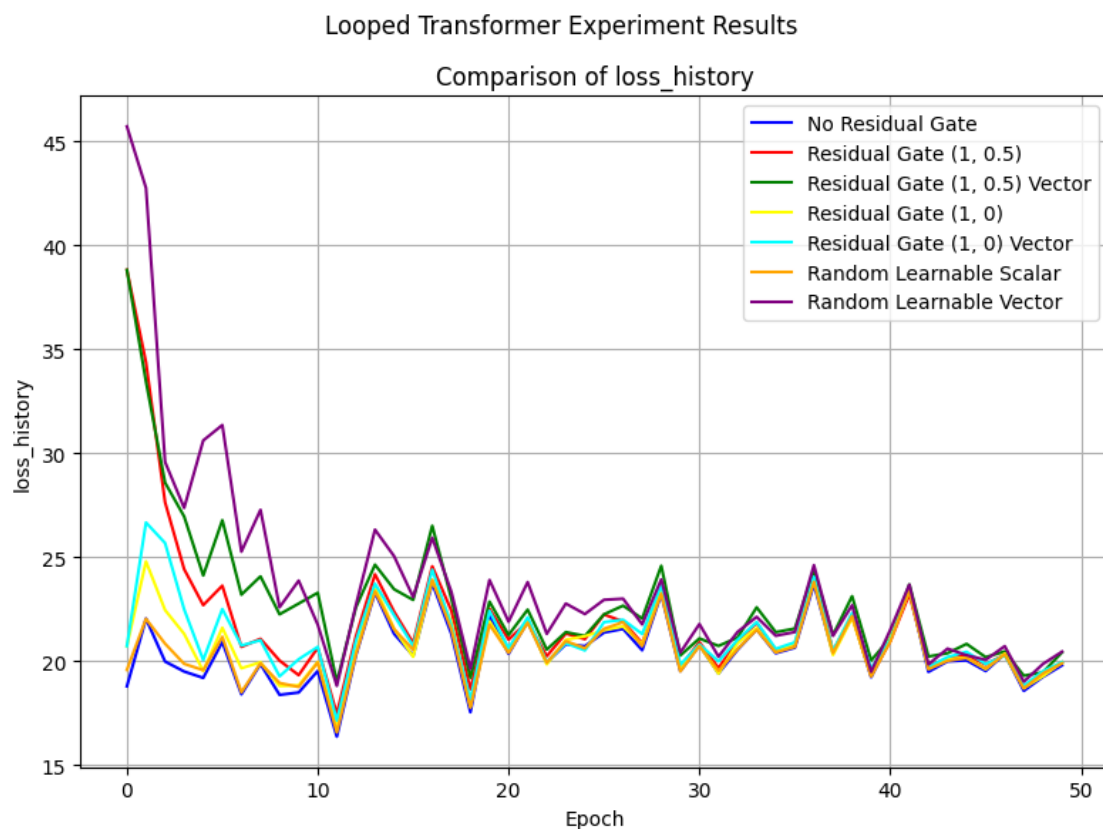
Epoch 50/50, Loss: 20.4417, current_blocks: 20

Training completed in 21.80 seconds.

[Experiment 7 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.16 GB

[Experiment 7 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB

Experiment 7/7 completed.



```
[26]: gate_table = ExperimentTable(params_groups=[
    {'experiment_name': 'Residual Gate (1, 0.5)', 'residual_gate': (1, 0.5),
    ↪ 'residual_gate_type': 'learnable_scalar'},
    {'experiment_name': 'Residual Gate (1, 0.5) Vector', 'residual_gate': (1, 0.
    ↪ 5), 'residual_gate_type': 'learnable_vector'},
    {'experiment_name': 'Residual Gate (1, 0)', 'residual_gate': (1, 0),
    ↪ 'residual_gate_type': 'learnable_scalar'},
    {'experiment_name': 'Residual Gate (1, 0) Vector', 'residual_gate': (1, 0),
    ↪ 'residual_gate_type': 'learnable_vector'},
    {'experiment_name': 'Random Learnable Scalar', 'residual_gate': 'random',
    ↪ 'residual_gate_type': 'learnable_scalar', 'residual_random': (0.9, 0.1)},
    {'experiment_name': 'Random Learnable Vector', 'residual_gate': 'random',
    ↪ 'residual_gate_type': 'learnable_vector', 'residual_random': (0.9, 0.1)},
], manual={'gate_lr_ratio': 100, 'scheduler_type': 'cosine', 'epochs': 100,
    ↪ 'print_every': 25})
gate_table.run(result_lists=[
    (['residual_gate_history_a_mean_relative',
    ↪ 'residual_gate_history_b_mean_relative'], 'epoch')
])
gate_table.plot(figure_size=(20,30),compare_experiments=False,
    ↪ subplot_shape=(-1,2), suptitle='')
```

```
Residual Gate (1, 0.5) initialized in 0.02 seconds. Using device: mps
Residual Gate (1, 0.5) Vector initialized in 0.01 seconds. Using device: mps
Residual Gate (1, 0) initialized in 0.01 seconds. Using device: mps
Residual Gate (1, 0) Vector initialized in 0.01 seconds. Using device: mps
Random Learnable Scalar initialized in 0.01 seconds. Using device: mps
Random Learnable Vector initialized in 0.01 seconds. Using device: mps
[本底显存检测] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.02 GB
[Experiment 1 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.02 GB
Epoch 25/100, Loss: 21.1192, current_blocks: 20
Epoch 50/100, Loss: 19.8862, current_blocks: 20
Epoch 75/100, Loss: 21.3765, current_blocks: 20
Epoch 100/100, Loss: 20.4947, current_blocks: 20
Training completed in 34.69 seconds.
[Experiment 1 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.16 GB
```

[Experiment 1 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB
Experiment 1/6 completed.

[Experiment 2 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB
Epoch 25/100, Loss: 21.3562, current_blocks: 20
Epoch 50/100, Loss: 20.4467, current_blocks: 20
Epoch 75/100, Loss: 21.5648, current_blocks: 20
Epoch 100/100, Loss: 20.8265, current_blocks: 20
Training completed in 35.89 seconds.

[Experiment 2 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.15 GB
[Experiment 2 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB
Experiment 2/6 completed.

[Experiment 3 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB
Epoch 25/100, Loss: 21.0529, current_blocks: 20
Epoch 50/100, Loss: 19.8676, current_blocks: 20
Epoch 75/100, Loss: 21.1713, current_blocks: 20
Epoch 100/100, Loss: 20.4512, current_blocks: 20
Training completed in 35.34 seconds.

[Experiment 3 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.15 GB
[Experiment 3 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB
Experiment 3/6 completed.

[Experiment 4 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB
Epoch 25/100, Loss: 20.5109, current_blocks: 20
Epoch 50/100, Loss: 20.1726, current_blocks: 20
Epoch 75/100, Loss: 21.4211, current_blocks: 20
Epoch 100/100, Loss: 20.5051, current_blocks: 20
Training completed in 36.48 seconds.

[Experiment 4 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.15 GB
[Experiment 4 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB
Experiment 4/6 completed.

[Experiment 5 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB
Epoch 25/100, Loss: 20.5986, current_blocks: 20
Epoch 50/100, Loss: 19.8349, current_blocks: 20
Epoch 75/100, Loss: 21.2072, current_blocks: 20

Epoch 100/100, Loss: 20.3681, current_blocks: 20

Training completed in 37.95 seconds.

[Experiment 5 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.16 GB

[Experiment 5 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB

Experiment 5/6 completed.

[Experiment 6 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB

Epoch 25/100, Loss: 22.4328, current_blocks: 20

Epoch 50/100, Loss: 20.2457, current_blocks: 20

Epoch 75/100, Loss: 21.3175, current_blocks: 20

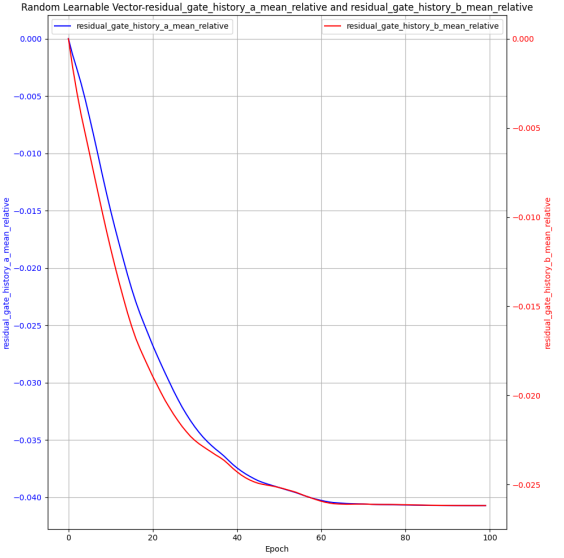
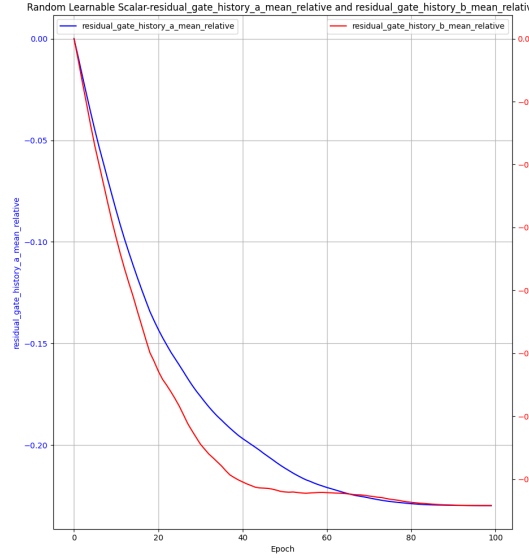
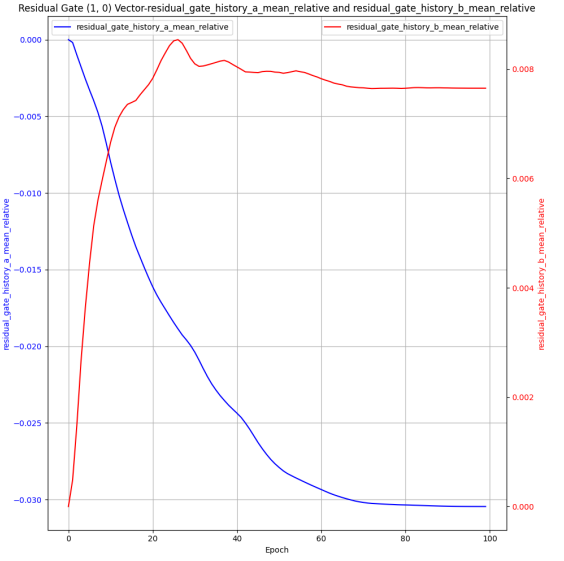
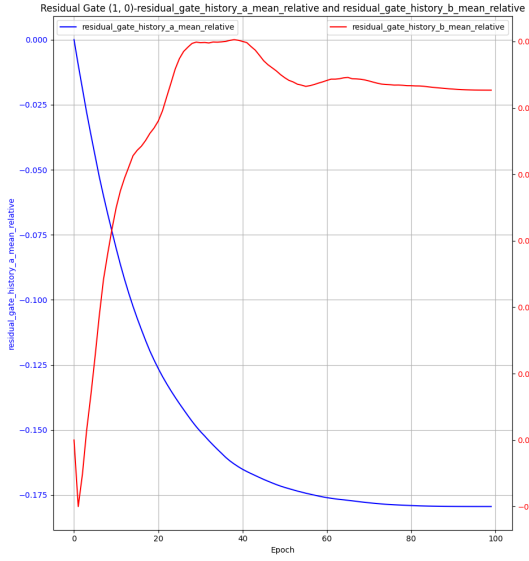
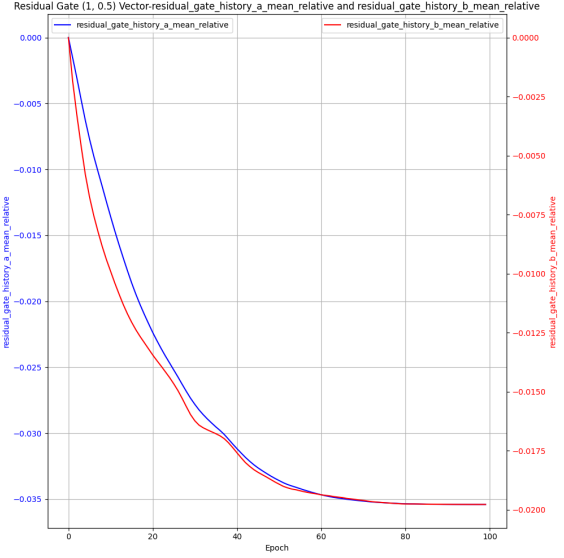
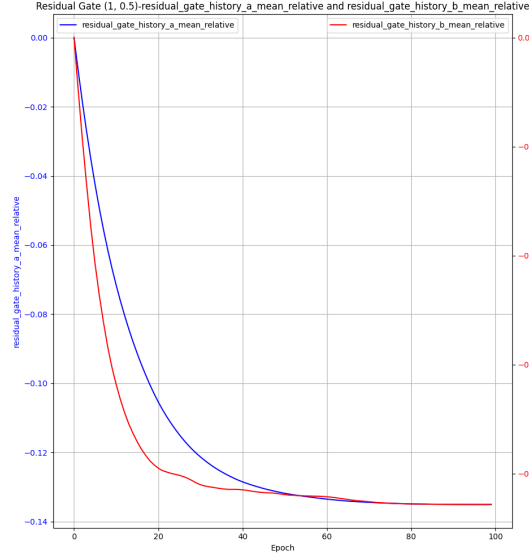
Epoch 100/100, Loss: 20.7065, current_blocks: 20

Training completed in 41.02 seconds.

[Experiment 6 训练峰值] 显存占用 -> PyTorch 分配: 0.03 GB | Metal 驱动分配: 2.16 GB

[Experiment 6 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.03 GB

Experiment 6/6 completed.



2.2.5 不同 scheduler 对比实验

```
[28]: experiment_table = ExperimentTable(params_groups=[{'experiment_name': 'No_
    ↪Scheduler'},
                                                    {'scheduler_type': 'cosine',
    ↪'experiment_name': 'Cosine Scheduler', 'lr_scale': 0},
                                                    {'scheduler_type': 'step',
    ↪'experiment_name': 'Step Scheduler', 'lr_scale': 0.1, 'step_size_scheduler':
    ↪90}],
    manual={'epochs':200, 'lr': 1e-3, 'optimizer_type': 'adamw', 'pe_type':
    ↪['alibi'],
            'wd_adamw': 0.2, 'd_model': 32, 'num_heads': 1, 'max_seq_len': 500,
    ↪'seq_len': 400, 'batch_size': 64, 'd_x': 20,
            'num_blocks': 20, 'num_eff': 15,
            'residual_gate': (1,1), 'residual_gate_type': 'learnable_scalar',
    ↪'gate_lr_ratio': 100,
            'scheduler_type': None, 'lr_scale': 0.01, 'step_size_scheduler': 10,
            'print_every': 50,
            'layer_weight_decay': 0.8, 'seq_weight_decay': 0.9
    })
experiment_table.run(result_lists=[(['loss_history'], 'epoch', 0),
    ↪(['y_pred_norm_history'], 'epoch', 0), (['y_true_norm_history'], 'epoch')])
experiment_table.plot(figure_size=(18, 6))
```

No Scheduler initialized in 0.02 seconds. Using device: mps

Cosine Scheduler initialized in 0.01 seconds. Using device: mps

Step Scheduler initialized in 0.01 seconds. Using device: mps

[本底显存检测] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB

[Experiment 1 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.04 GB

Epoch 50/200, Loss: 19.8760, current_blocks: 20

Epoch 100/200, Loss: 21.0343, current_blocks: 20

Epoch 150/200, Loss: 19.6615, current_blocks: 20

Epoch 200/200, Loss: 18.4375, current_blocks: 20

Training completed in 66.28 seconds.

[Experiment 1 训练峰值] 显存占用 -> PyTorch 分配: 0.02 GB | Metal 驱动分配: 2.13 GB
[Experiment 1 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.07 GB
Experiment 1/3 completed.

[Experiment 2 训练前] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.08 GB
Epoch 50/200, Loss: 19.8793, current_blocks: 20
Epoch 100/200, Loss: 21.0404, current_blocks: 20
Epoch 150/200, Loss: 19.6585, current_blocks: 20
Epoch 200/200, Loss: 18.4290, current_blocks: 20
Training completed in 64.98 seconds.
[Experiment 2 训练峰值] 显存占用 -> PyTorch 分配: 0.02 GB | Metal 驱动分配: 2.13 GB
[Experiment 2 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.08 GB
Experiment 2/3 completed.

[Experiment 3 训练前] 显存占用 -> PyTorch 分配: 0.01 GB | Metal 驱动分配: 0.09 GB
Epoch 50/200, Loss: 19.8760, current_blocks: 20
Epoch 100/200, Loss: 21.0657, current_blocks: 20
Epoch 150/200, Loss: 19.6840, current_blocks: 20
Epoch 200/200, Loss: 18.4393, current_blocks: 20
Training completed in 65.11 seconds.
[Experiment 3 训练峰值] 显存占用 -> PyTorch 分配: 0.02 GB | Metal 驱动分配: 2.13 GB
[Experiment 3 清理后] 显存占用 -> PyTorch 分配: 0.00 GB | Metal 驱动分配: 0.08 GB
Experiment 3/3 completed.



3 Non-linear Regression Experiments

3.1 实验数据观测

```
[30]: manual={ 'data_type': 'nonlinear',  
               'function_callable': lambda x: F.relu(x),  
               'max_seq_len': 500,  
               'scheduled_training': True,  
               'pe_type': ['alibi'],  
               'residual_gate': (1, 0.5),  
               'residual_gate_type': 'learnable_scalar',  
               'num_eff': 5,  
               'num_blocks': 20,  
               'batch_size': 8,  
               'num_heads': 2,  
               'd_model': 128,  
               'seq_len': 400,  
               'd_hidden': 8,  
               'epochs': 100}
```

```
[31]: loss_table = ExperimentTable(params_groups=[{'experiment_name': 'loss_history',  
                                                  'epochs': 500,  
                                                  'print_every': 100}],  
                                   ↪manual=manual)  
loss_table.run(result_lists=[(['loss_history'], 'epoch'),  
                              ↪(['y_norm_ratio_history', 'residual_gate_history_a',  
                              ↪'residual_gate_history_b'], 'epoch')])  
loss_table.plot.figure_size=(8,12), compare_experiments=False,  
↪subplot_shape=(-1,1))
```

loss_history initialized in 0.02 seconds. Using device: mps

[本底显存检测] 显存占用 -> PyTorch 分配: 0.22 GB | Metal 驱动分配: 0.28 GB

[Experiment 1 训练前] 显存占用 -> PyTorch 分配: 0.22 GB | Metal 驱动分配: 0.28 GB

Epoch 100/500, Loss: 0.4739, Norm Ratio: 0.4741, Current Blocks: 20

Epoch 200/500, Loss: 0.3862, Norm Ratio: 0.6557, Current Blocks: 20

Epoch 300/500, Loss: 0.6595, Norm Ratio: 0.5505, Current Blocks: 20

Epoch 400/500, Loss: 0.3205, Norm Ratio: 0.5234, Current Blocks: 20

Epoch 500/500, Loss: 0.3823, Norm Ratio: 0.5465, Current Blocks: 20

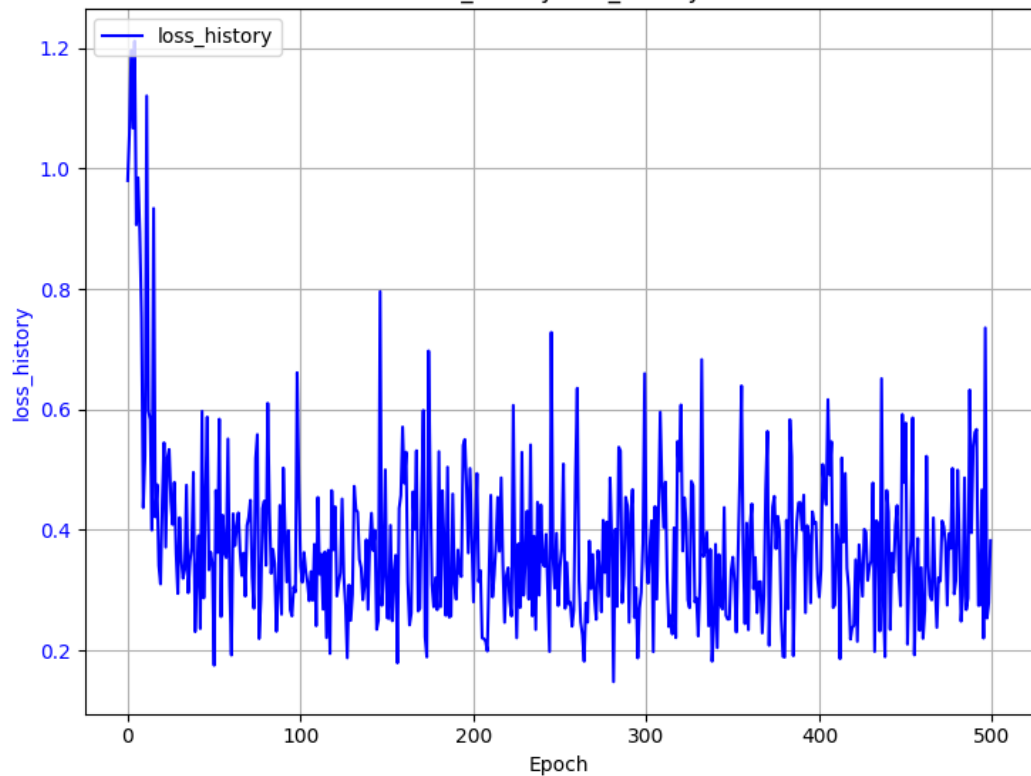
Training completed in 51.75 seconds.

[Experiment 1 训练峰值] 显存占用 -> PyTorch 分配: 0.23 GB | Metal 驱动分配: 0.93 GB

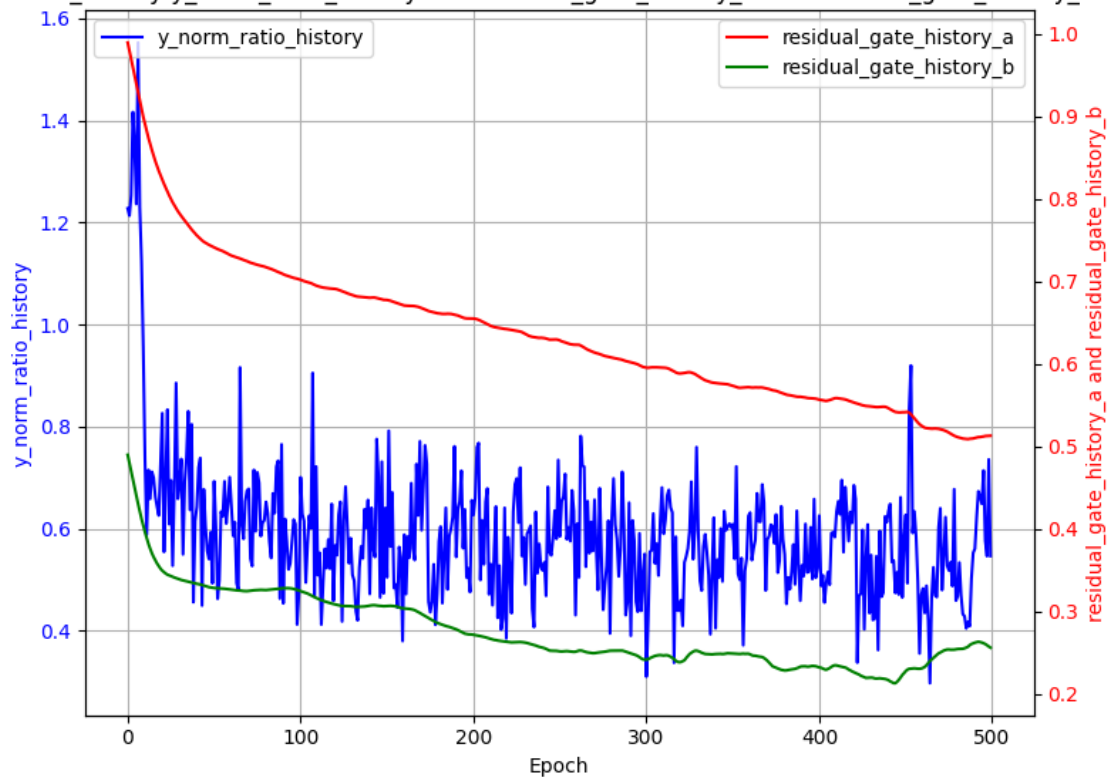
[Experiment 1 清理后] 显存占用 -> PyTorch 分配: 0.22 GB | Metal 驱动分配: 0.29 GB

Experiment 1/1 completed.

Looped Transformer Experiment Results
loss_history-loss_history



loss_history-y_norm_ratio_history and residual_gate_history_a and residual_gate_history_b



实验数据分析 1. 参数量分析 2. norm_ratio 与 loss 的关系分析 - 对于 Linear+ReLU+Linear，由于 Linear 矩阵归一化后方差为 1，ReLU 又会丢掉一半的激活，因此 y_true 的二阶矩（能量）期望为 $\mathbb{E}[y^2] \approx 0.5$ - 模型在进行 In-Context Learning (ICL) 时，遇到了认知瓶颈。它把复杂的真实信号 y 拆解成了两个部分： $y = y_{learnable} + y_{noise}$ - $y_{learnable}$ ：模型通过现有的上下文能够学明白的“低频/线性”规律。- y_{noise} ：由于非线性过于复杂或上下文信息仍显不足，模型无法解开的“高频/非线性”残差。- 在 MSE 损失函数的逼迫下，模型最聪明的做法就是精准预测自己能看懂的部分，对看不懂的部分直接输出 0。因此它的预测值 $\hat{y} \approx y_{learnable}$ 。- 根据 Norm Ratio 倒推 loss：- 观察到 Norm Ratio 在 0.4 附近震荡，即

$$NormRatio = \sqrt{\frac{\mathbb{E}[\hat{y}^2]}{\mathbb{E}[y^2]}} \approx \sqrt{\frac{\mathbb{E}[y_{learnable}^2]}{\mathbb{E}[y_{learnable}^2] + \mathbb{E}[y_{noise}^2]}} \approx 0.4$$

- 代入 $\mathbb{E}[y^2] \approx 0.5$ ，可以推断出 $\mathbb{E}[y_{learnable}^2] \approx 0.08$ ， $\mathbb{E}[y_{noise}^2] \approx 0.42$ 。- 由于 $\hat{y} \rightarrow y_{learnable}$ ，因此最终的 loss 约为

$$\mathbb{E}[(y - \hat{y})^2] = \mathbb{E}[y_{noise}^2] + \mathbb{E}[(y_{learnable} - \hat{y})^2] \approx 0.42$$

与实际观察到的 loss 值相符。